

Now , we will explore How to Create a RAP Application (Unmanaged Scenario)

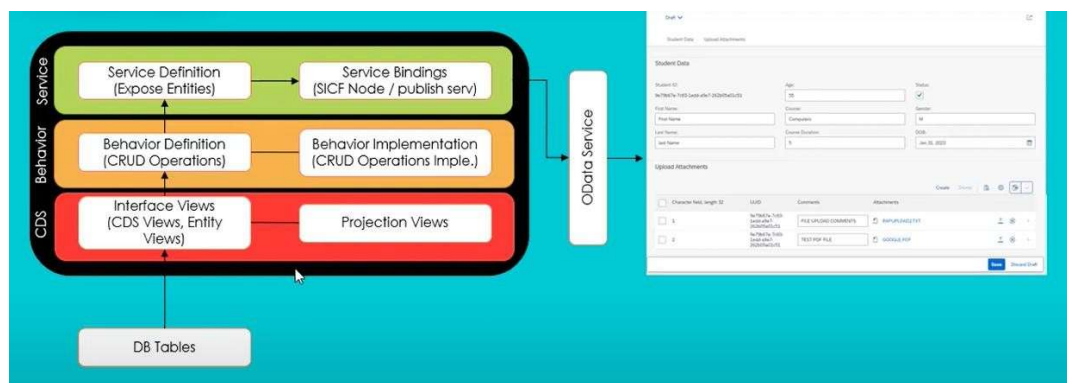
The difference between Managed and Unmanaged scenario is that the former lets SAP handle the CRUD operations and in later (Unmanaged) we write our custom code to manage all the operations.

From all practical standpoint, we would be using Unmanaged Scenario, so it is essential to understand it.

Part 1 – Create the data model i.e. Root and Projection CDS Entities, Metadata Extensions

Part 2 – Create Behavior Definition, Business Service Definitions and Binding and Test the application display functionality

Part 3 – Implement Behavior for the Parent & Child Entity



When you create a behavior definition in an unmanaged scenario, it generates a class with two sections: **Handler** and **Saver**. These classes are easily identified by their prefixes: **LHC_** for the **Handler** and **LSC_** for the **Saver**.

Key Points:

1. Handler Class (LHC_):

- **Function:** This class handles the logic related to transactions. It works on the **transactional buffer**, which is temporary storage.
- **Role:** It processes incoming data from the frontend but does **not save** it to the actual database. It just holds the data temporarily in the buffer.

2. Saver Class (LSC_):

- **Function:** This class is triggered when the logic needs to **save** the data to the database.
- **Role:** It moves the data from the **transactional buffer** to the **persistent state** (actual database), thus saving the data permanently.

ABAP RAP Class (Un-Managed) : Behavior Definition with Unmanaged option selected.



Key Components of Derived Types:

To implement CRUD operations in an unmanaged scenario, it is important to understand the **components of Derived Types and their use**. These components are essential for managing data and handling operations such as Create, Read, Update, and Delete within the application.

Derived Types:

- **%cid (Context ID)**: Unique identifier for a session or transaction, used to track activities.
- **%cid_ref (Context ID Reference)**: Link to a specific Context ID, connecting related data across processes.
- **%tky (Token)**: Security key to authenticate and validate sessions or requests.
- **%is_draft (Is Draft Flag)**: Indicates whether the data is still in draft or finalized state.
- **%pid (Parent ID)**: Links a child entity to its parent, used in hierarchical data structures.
- **%param (Parameter)**: A value passed into a function to control how it behaves.
- **%control (Control Flag)**: A flag to guide the flow of a process or trigger specific actions.
- **%fail (Failure Flag)**: Indicates if an operation failed, used for error handling.
- **%msg (Message)**: Stores messages, typically for logging or error reporting.

Component of the Derived Types	
Data Type	
%cid	%control
%cid_ref	%fail
%tky	%msg
%is_draft	
%pid	
%param	

ABAP RAP EML.

- EML stands for entity manipulation language and it's kind of a new language. it's also a part of ABAP languages or subset of ABAP language. Using which user can access the RAP Business object or classes.
- Using EML we can make changes to the data of BO, we can create data, modify , edit , update , delete so all Crud operations are quite possible using EML.

Below is main syntax while using EML.

Syntax for use to modify the BO.

EML in RAP: EML is a part of ABAP and using which user can access the data of RAP business objects in an ABAP program.
EML statements allow the data content of a RAP BO to be read or modified.

```
MODIFY ENTITIES OF <ROOT ENTITY NAME>
ENTITY <ENTITY ALIAS NAME>
<OPERATION NAME> FROM <INTERNAL TABLE NAME WHICH CONTAINS DATA>
MAPPED DATA(<MAPPED TABLE NAME>)
FAILED DATA(<FAILED TABLE NAME>)
REPORTED DATA(<REPORTED TABLE NAME>).
```

<OPERATION NAME> → CREATE, UPDATE, DELETE

- Words are in **Blue colours** are all Standard words or keywords / Constant and in **yellow colours** they will keep changing according the BO .
- First line says **Modify entities of** is keyword and **Root entity** name is you root entity (ZR) .
- Second line says **Entity** is keyword and then provide the **root entity Alias name** which probably you define in your behavior definition.
- Third line it says **Operation name** so operation name could be your create update or delete then **from** and then you provide **Internal table** here which contains your data for example if you want to create something your internal table will have all the data.
- **Mapped data , failed data and reported data** so these are all the written parameter for the modify.
- **Mapped data** : we Define an internal table here which will contain your key fields or whichever is created newly.
- **Failed data** : the operation which you are performing in these three if that operation fails then this internal table will contain your failed information .
- **Reported data** : this mainly used when you want to show any error message to the front end .

These are all three the returning parameter for modify entity statement.

ABAP EML Response Parameters :

Response Parameters are used along with the EML Statements to get Information about Statement Execution result.

Response Parameters	Response Parameters Used for
FAILED (LT_FAILED)	Contains details about operation which is not successful or that could not be processed
MAPPED (LT_MAPPED)	Contains Key Mapping Information (contains table Key value). Only available for MODIFY ENTITY OR MOFIDY ENTITIES
REPORTED (LT_REPORTED)	Contains any Error Messages as result of Failed Operation

EML commit entities :

ABAP EML COMMIT ENTITIES : As a result of any MODIFY statement data doesn't go in Persistent state and it is available in Transactional Buffer until COMMIT is triggered.

COMMIT ENTITIES → Triggers ABAP RAP framework SAVE sequence. RAP BO is changed using MODIFY statement is saved in Transactional Buffer.

The saving of changed or created data to the database must be triggered, which is done by COMMIT ENTITIES statements.



- **Commit entities** is a statement which is a result of any modify statement which seen in create , delete and update the data doesn't go in the persistent State and it is available in transactional buffer until the commit is triggered .
- **Commit entity** is triggers a library framework save sequence and business object is changed using modify statement is saved in the transactional buffer.
- Whatever changes you have made in any of the rap business object those will be saved in the transactional buffer.

ABAP EML Create And Create By Association :

Create By Association: Create By Association is used when there is need to create Instances based on other Instances. This type of creation used when there is an Associations especially in case of Composition. In this scenario **Create By Association** is very Important because entity linked with Composition can not exists alone (Order Header and Order Item).

```
MODIFY ENTITIES OF <ROOT ENTITY NAME>
ENTITY <ENTITY ALIAS NAME> OR <ROOT ENTITY NAME>
CREATE FROM <HEAD INTERNAL TABLE NAME WHICH CONTAINS DATA>
CREATE BY \_<ASSOCIATION NAME> FROM <ITEM INTERNAL TABLE NAME WHICH CONTAINS DATA>
MAPPED DATA(<MAPPED TABLE NAME>)
FAILED DATA(<FAILED TABLE NAME>)
REPORTED DATA(<REPORTED TABLE NAME>).
```

- **Blue colour** these are all the keywords and in the **Yellow colour** these are all the values which will be changed based on the business object.
- **Create form** here will have header internal table name , which contains header internal table data.
- **Create by** is the association name which mentioned or maintained while creating the CDS View ,so the same Association name will be here , but you need to put a **backslash(\)** (Do ctrl+space you will get association name automatically) **From<>** item internal table name which contains the data of items or the association data.
- Rest of the three things are the response parameters.

Derived Data Types:

Data Type	Used for
%CID	%CID – ContentID is temporary Primary Key for an Instance, its valid till actual Primary key is not generated by runtime.
%CID_REF	%CID_REF – Specifies reference to ContentID. If need to refer header and Child record then %CID_REF is populated with header %CID value.

1. %CID

- %CID and which stands for Content ID
- Content ID is a temporary primary key for an instance and it is valid till the actual primary key is generated by the runtime .

2. %CID_REF

- Mainly used when create by association or child in header entities relationship kind of operation.
- %CID_REF this specifies the reference to the control ID so this will always have value which maintained in %CID.
- So if need to refer the header and child record then % CID_REF is populated with header %CID value.

Note : Whatever %CID value you will maintain for header record that should be same maintained for %CID_REF so that these two records can be connected together.

ABAP EML Read Entity , Entities and By Association

1. Read Entity

ABAP EML READ ENTITY:

READ ENTITY → Used to perform read-only operations on RAP BO instances. This is also called short form of the READ statement, which is used for read operations from a single entity only.

- The entity name (entity) can be specified directly.
- Reading from a Child Entity can also be done directly.
- Entity Alias name can not be used with Read Entity.

```
READ ENTITY <ROOT ENTITY NAME>
ALL FIELDS WITH VALUE #( ( %key-<KEY FIELD> = <KEY FIELD VALUE TO READ DATA> ) )
RESULT DATA (<RESULT TABLE NAME>)
FAILED DATA(<FAILED TABLE NAME>).
```

It is only used to perform the read-only operation on BO entities and this is also called a **short form** of the read statement. which is used to read the operation for a single entity only , it cannot be used for read multiple entities.

If you want to read a single entity then you can go with the read entity statement .

- **Blue Colours** are the keywords and **Yellow Colours** are text which will be changing based on the BO.
- **Read entity** is root entity name or Child entity name .
- Need to read or get all of the fields and then with **Value** will provide the key field and key field name, the value data finally available in the result.
- If result is failed then have **failed** .
- if read statement is successful then will get the data into the **result** and if it fails then data will be in **failed** table.

2. Read Entities and Association

ABAP EML READ ENTITIES:

READ ENTITIES OF → The long form of the READ statement is used for collecting multiple read operations on multiple entities.

- It can also be used to just read from a single entity.
- Root Entity must be used first in statement and then Individual Entity (E1, E2.. En) can be used.
- All other entities must be entities contained in the composition tree of the root entity.

- Read entities suggest that it has read multiple entities using **read entities off Syntax** so this is also called as **long form** of read statement and it is used for collecting the multiple read operations.
- while using the read entities statement , **root entities** must be used in the **first statement** of your read and after that you can use the **individual entities** whatever entities you want to use in your read entity statement it can be **Association** .
- The only difference between **read entity** and **read entities off** is in read entities you can read multiple entities basically the collection of multiple reads and in read entity we use single read.

Syntax Of Read Entities :

First read the root entity here and then, read other entity , Associated entity.

```

READ ENTITIES OF <ROOT ENTITY NAME>
ENTITY <alias> OR <ROOT ENTITY NAME>
ALL FIELDS WITH VALUE #( ( %key-<KEY FIELD> = <KEY FIELD VALUE TO READ DATA> ) )
RESULT DATA (<RESULT TABLE NAME>)

ENTITY <ENTITY NAME>
ALL FIELDS WITH VALUE #( ( %key-<KEY FIELD> = <KEY FIELD VALUE TO READ DATA> ) )
RESULT DATA (<RESULT TABLE NAME>)

ENTITY <Root Entity alias>
BY \_<ASSOCIATION NAME>
ALL FIELDS WITH VALUE #( ( %key-<KEY FIELD> = <KEY FIELD VALUE TO READ DATA> ) )
RESULT DATA (<RESULT TABLE NAME>)
FAILED DATA(<FAILED TABLE NAME>).

```

ABAP EML Dynamic form

MODIFY ENTITIES OPERATIONS, Dynamic Form:

MODIFY ENTITIES OPERATIONS → The dynamic version of the **MODIFY ENTITIES** statement executes multiple modify operations for multiple RAP BOs in single **MODIFY** statement.

op_tab must be a table of type **ABP_BEHV_CHANGES_TAB**. This table has specific components:

Component	Description / Use
op	It is a mandatory component specifying the operation to be executed
entity_name	Name of the entity for which the operation is executed. The name must be provided using capital letters.
instances	It is a mandatory component specifying the instances in an internal table.

Data Model and Business Object

The Student Details business object consists of two entities that are structured in a hierarchy tree. Every entity in the BO-composition tree is model with a CDS view entity. The business entities we are going to work on in our present scenarios are:

Student: The root entity defines general student data like the Student ID or Name.

Academic: The academic entity defines academic details like Course , Semester , Result.

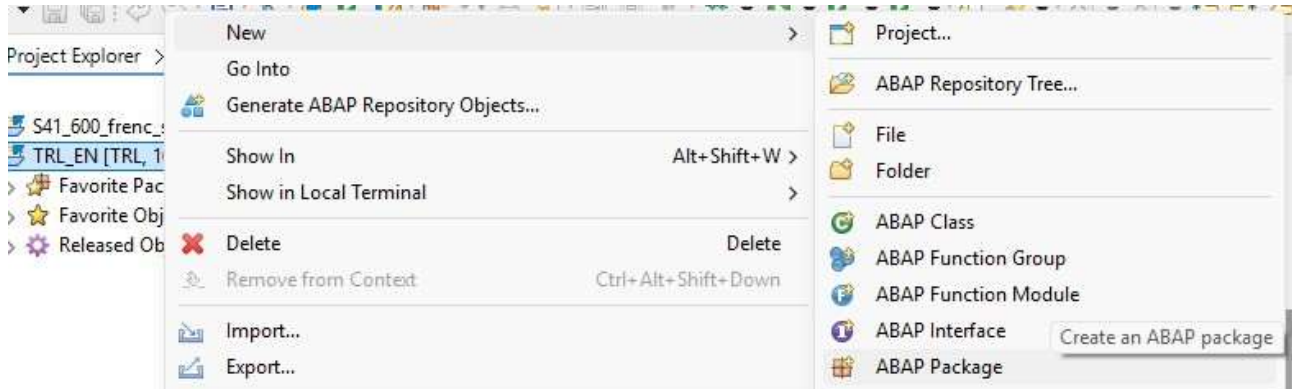
Prerequisites

- You need an SAP BTP, ABAP environment trial user or a license.
- You have downloaded and installed the latest ABAP Development Tools (ADT).
- ABAP Flight Reference Scenario must be available in your ABAP system. You can download the complete reference scenario from GitHub: [Downloading the ABAP Flight Reference Scenario](#).

Build the data model .

1. Create Package

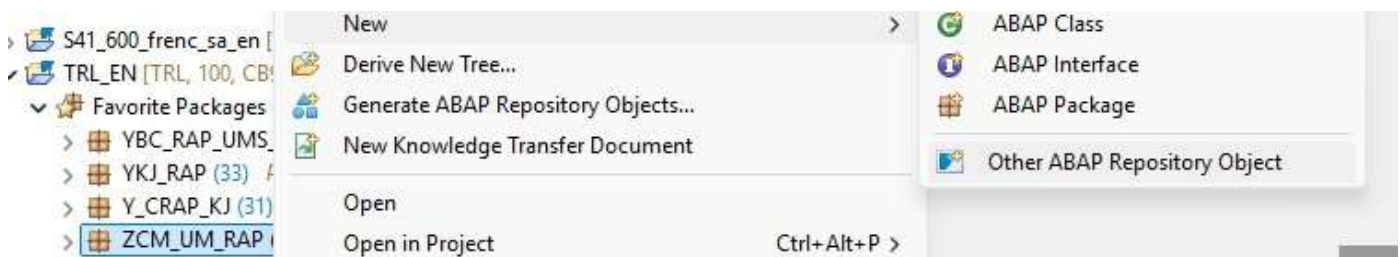
The first thing that needed to be done was to create an ABAP package, this is quite simple. Simply just log into the SAP system. From there, select the option to create new package. The system prompts to provide a name and description for the package. Don't forget to check Add to favorite packages so that you can access it easily.



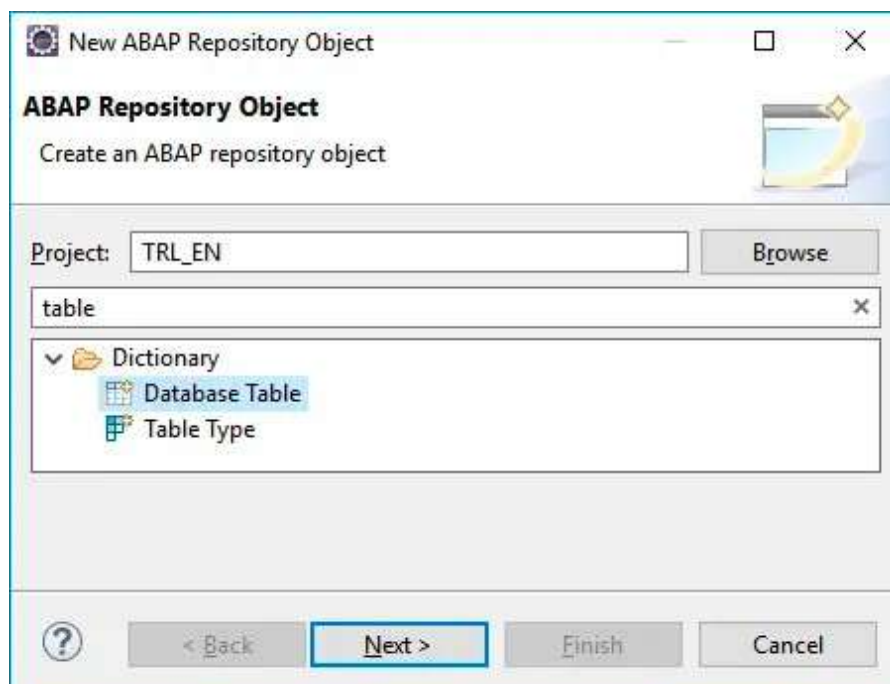
Step 2 . Database Table

Create Table for Student Data

1. Right click on the package and choose **New>Other ABAP Repository Object**.



2. Choose **Database Table**, Click **Next**.



3. Enter **Name** and **Description** . click on next select **TR** and click **Finish and below in table.**

4. Complete table with below code.

```
@EndUserText.label : 'Student Detail Table'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table zstudent_tb {

  key client      : abap.clnt not null;
  key id          : sysuuid_x16 not null;
  studentid      : ze_studentid;
  firstname      : abap.char(100);
  lastname       : abap.char(100);
  age            : abap.char(4);
  course         : abap.char(50);
  courseduration : abap.char(4);
  status         : abap_boolean;
  gender         : abap.char(1);
  dob            : abap.dats;
  lastchangedat  : timestamp;
  locallastchangedat : timestamp;

}
```

Note : Need to Create Data Element and Domain For Student ID.

Data Domain :

The screenshot shows the 'Domain: ZD_STUDENTID' configuration window. It has two tabs: 'Format' and 'Output Characteristics'. In the 'Format' tab, the 'Data Type' is set to 'NUMC' and the 'Length' is set to '5'. There is a 'Browse...' button next to the Data Type field. In the 'Output Characteristics' tab, the 'Output Length' is set to '5' and there is a 'Conversion Routine' field.

Data element :

The screenshot shows the 'Data Element: ZE_STUDENTID' configuration window. It has two tabs: 'Data Type Information' and 'Field Labels'. In the 'Data Type Information' tab, the 'Category' is set to 'Domain', the 'Type Name' is 'ZD_STUDENTID', the 'Data Type' is 'NUMC', and the 'Length' is '5'. There is a 'Browse...' button next to the Type Name field. In the 'Field Labels' tab, there are four rows for 'Short', 'Medium', 'Long', and 'Heading' labels, each with a corresponding length field set to 10, 20, 40, and 55 respectively.

Table Explanation:

Annotations:

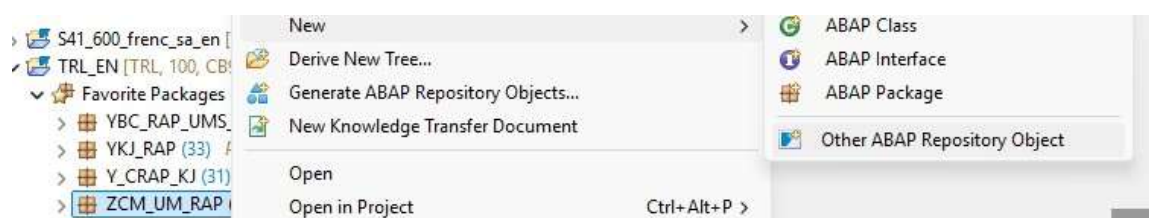
- **@EndUserText.label** : 'Student Detail Table': Label for the table, indicating it stores student details.
- **@AbapCatalog.enhancement.category** : #NOT_EXTENSIBLE: The table cannot be extended (no additional fields can be added after creation).
- **@AbapCatalog.tableCategory** : #TRANSPARENT: The table directly maps to a database table.
- **@AbapCatalog.deliveryClass** : #A: The table holds application data and can be transported between systems.
- **@AbapCatalog.dataMaintenance** : #RESTRICTED: Data maintenance is restricted (it cannot be directly maintained via standard transactions like SM30).

Fields:

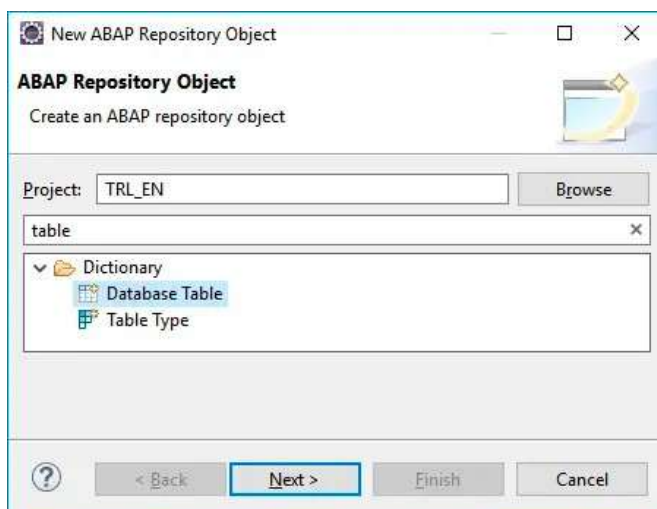
- **client**: Client ID (mandatory).
- **id**: Unique student identifier (UUID, mandatory).
- **lastchangedat**: Timestamp of the last change.
- **locallastchangedat**: Local timestamp of the last change.

Create Table for Student Academic Data

1. Right click on the package and choose **New>Other ABAP Repository Object**.



2. Choose **Database Table**, Click **Next**.



3. Enter **Name** and **Description**. click on next select **TR** and click **Finish** and below in table.

4. Complete table with **below code**.

```

1 @EndUserText.label : 'table for academic'
2 @AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
3 @AbapCatalog.tableCategory : #TRANSPARENT
4 @AbapCatalog.deliveryClass : #A
5 @AbapCatalog.dataMaintenance : #RESTRICTED
6 define table zacademic_tb {
7
8     key client : abap.clnt not null;
9     @AbapCatalog.foreignKey.screenCheck : false
10    key id      : sysuuid_x16 not null
11        with foreign key zstudent_tb
12            where client = zacademic_tb.client
13                and id = zacademic_tb.id;
14    key course : ze_course not null;
15    key sem    : ze_sem not null;
16    semresult  : ze_semr;
17
18 }

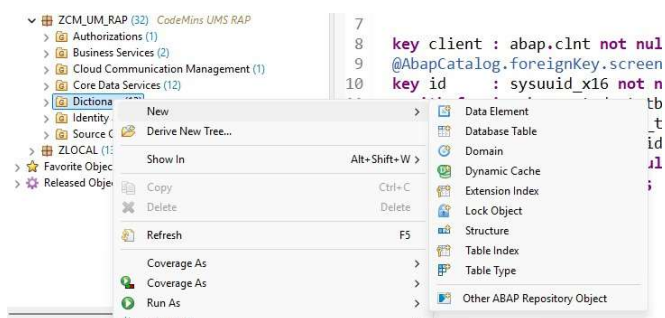
```

5. Here Need to add **Data Elements of Course, Semester and Semester Result** in table Follow Below Procedure.

Step1.Create Domain Course , Semester and Semester Result

1.Course

1. right click on **Dictionary>Data Domain**.



2. Provide **Name** and **Description** . Click on **Next** . Select **TR** . **Finish**

The screenshot shows the 'New Domain' dialog box. The 'Domain' section is active, showing 'Create Domain'. The 'Project' field is set to 'TRL_EN' and the 'Package' field is set to 'ZCM_UM_RAP'. The 'Name' field is set to 'ZD_COURSE' and the 'Description' field is set to 'Domain Course'. The 'Original Language' field is set to 'EN'. At the bottom, there are buttons for '< Back', 'Next >', 'Finish', and 'Cancel'.

3. Provide Data Type Activate like Below .

Domain: ZD_COURSE

Format

Data Type: CHAR

Length: * 1

Output Characteristics

Output Length: 1

Conversion Routine:

☐ Case-sensitive

Fixed Values

Define the single fixed values or interval of values

☐ Intervals

type filter text

Fixed Value	Description
C	Computer
E	Electronics
<Enter new value>	

2. Semester

Same Process follow like course.

Domain: ZD_SEM

Format

Data Type: CHAR

Length: * 4

Output Characteristics

Output Length: 4

Conversion Routine:

☐ Case-sensitive

Fixed Values

Define the single fixed values or interval of values

☐ Intervals

type filter text

Fixed Value	Description
I	1st Sem
II	2nd Sem
III	3rd Sem
IV	4th Sem
V	5th sem
VI	6th Sem
<Enter new value>	

3. Semester Result

Same process follow like Course

Domain: ZD_SEMR

Format

Data Type: CHAR

Length: * 2

Output Characteristics

Output Length: 2

Conversion Routine:

☐ Case-sensitive

Fixed Values

Define the single fixed values or interval of values

☐ Intervals

type filter text

Fixed Value	Description
P	Pass
F	Fail
<Enter new value>	

2.Data Element

Process for data element : Right click on **Dictionary > Data Element > Name and Description >Next > Finish.**

Provide Domain Name in **Type Name And Activate** :

1.Course

Data Element: ZE_COURSE

Data Type Information		Field Labels	
Specify the data type of the data element		Provide field labels and set maximum lengths	
Category:	Domain	Short:	10
Type Name:	ZD_COURSE Browse...	Medium:	20
Data Type:	CHAR	Long:	40
Length:	1	Heading:	55

2. Semester

Data Element: ZE_SEM

Data Type Information		Field Labels	
Specify the data type of the data element		Provide field labels and set maximum lengths	
Category:	Domain	Short:	10
Type Name:	ZD_SEM Browse...	Medium:	20
Data Type:	CHAR	Long:	40
Length:	4	Heading:	55

► Additional Properties

3.Sem Result

Data Element: ZE_SEMR

Data Type Information		Field Labels	
Specify the data type of the data element		Provide field labels and set maximum lengths	
Category:	Domain	Short:	10
Type Name:	ZD_SEMR Browse...	Medium:	20
Data Type:	CHAR	Long:	40
Length:	2	Heading:	55

Foreign Key:

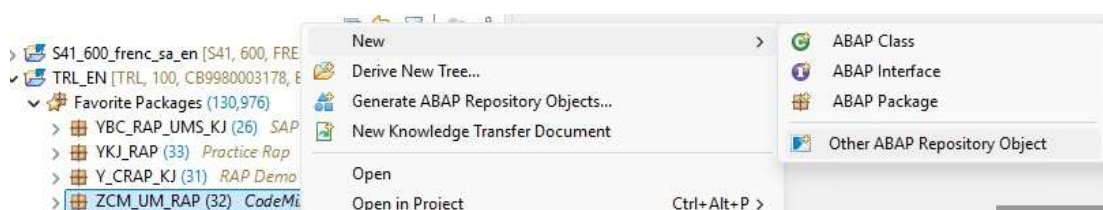
- **id** is a foreign key referencing **zstudent_tb** with a condition ensuring that both the **client** and **id** match between the two tables.

2.Create Root CDS Entities

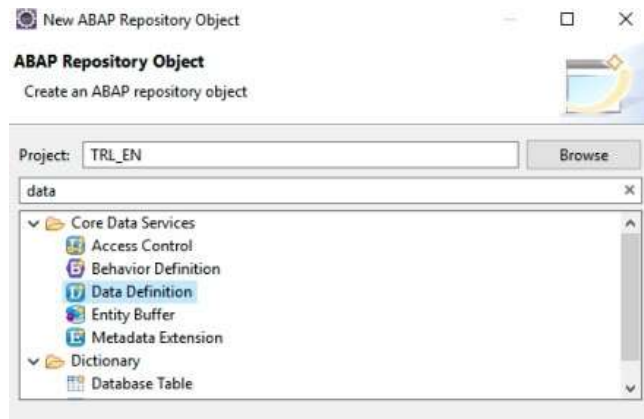
As we have a Student which consists of multiple student data– this forms a Parent-Child / Header-Item relationship which we will need to create in CDS Entities as well.

Create Root CDS Entity for Student Data

1. Right click on the **package->New->Other Repository Object**

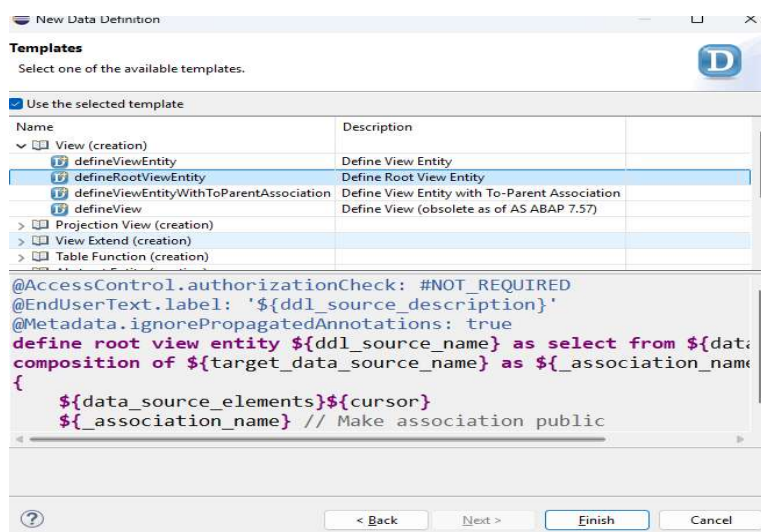


2. Select **Data Definition**, Click **Next**.



3. Provide **Name** and **Description** for the Entity and Click **next** . Select **TR** and click on **Next**.

Select Template as '**Define Root View Entity**' .Finish.



4. shown template where we need to complete with the code. **Complete with below Code**

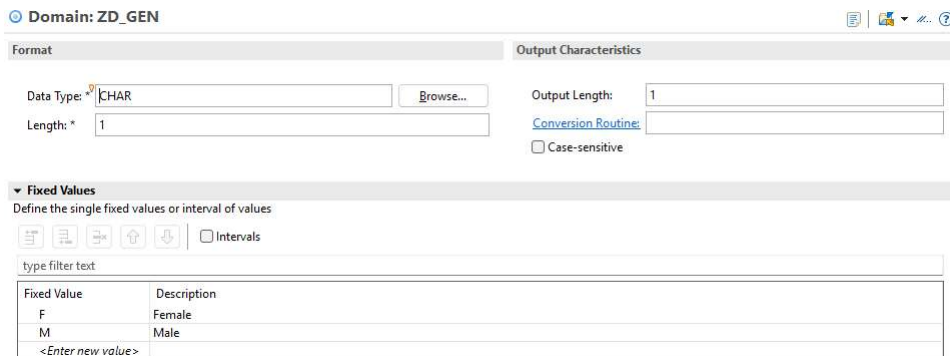
Complete the CDS Root View Entity with the below code. Save the Entity. Note that we have a composition that refers to another CDS View Entity which is not yet created. So, you will not be able to activate this yet. Both entities will have to be activated together.

```
1 @AccessControl.authorizationCheck: #NOT_REQUIRED
2 @EndUserText.label: 'Root entity for Student'
3 @Metadata.ignorePropagatedAnnotations: true
4 define root view entity ZR_STU
5   as select from zstudent_tb
6   composition[0..*] of ZI_Acad as _academic
7   association[1..*] to ZEI_GEN as _gender on $projection.Gender = _gender.Value
8 {
9   key id
10    studentid as StudentID,
11    firstname as Firstname,
12    lastname as Lastname,
13    age as Age,
14    course as Course,
15    courseduration as Courseduration,
16    status as Status,
17    gender as Gender,
18    _gender.Description as GenderDesc,
19    dob as Dob,
20    lastchangedat as Lastchangedat,
21    locallastchangedat as Locallastchangedat,
22    // _association_name
23    _academic,
24    _gender
25 }
26
```

Note : Here We adding Gender Description (When the user writes 'F' or 'M' in gender field, the gender description field should automatically display 'Female' and 'Male'.)

Procedure For Gender description :

1. Create Domain for gender Description.



Domain: ZD_GEN

Format

Data Type: CHAR

Length: 1

Output Length: 1

Conversion Routine:

Case-sensitive: ☐

Fixed Values

Define the single fixed values or interval of values

type filter text

Fixed Value	Description
F	Female
M	Male
<Enter new value>	

2. Create CDS Entity for Gender Description. (right click on core data service > Data Definition>Provide name and description>Next>Select TR >Choose Template as 'Define View Entity'>Finish)

Syntax for get domain value in CDS entity. Syntax : **DDCDS_CUSTOMER_DOMAIN_VALUE_T**(
p_domain_name:'ZD_GEN')

```

1= @AbapCatalog.viewEnhancementCategory: [#NONE]
2 @AccessControl.authorizationCheck: #NOT_REQUIRED
3 @EndUserText.label: 'Interface Entity Gender Desc.'
4 @Metadata.ignorePropagatedAnnotations: true
5 @ObjectModel.usageType: {
6   serviceQuality: #X,
7   sizeCategory: #S,
8   dataClass: #MIXED
9 }
10 define view entity ZEI_GEN as select from DDCDS_CUSTOMER_DOMAIN_VALUE_T
11 {
12   key domain_name,
13   key value_position,
14   @Semantics.language: true
15   key language,
16   value_low as Value,
17   @Semantics.text: true
18   text as Description
19 }
20

```

3. Activate Gender Description Entity and Add in Root Entity as Association _gender.

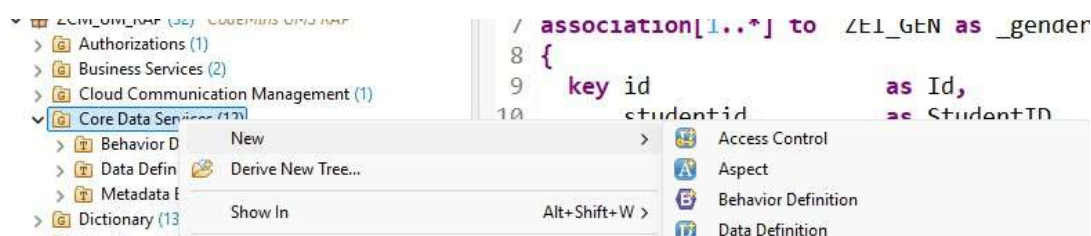
```

1 @AccessControl.authorizationCheck: #NOT_REQUIRED
2 @EndUserText.label: 'Root entity for Student'
3 @Metadata.ignorePropagatedAnnotations: true
4 define root view entity ZR_STU
5 {
6   as select from zstudent_tb
7   composition[0..*] of ZI_Acad as _academic
8   association[1..*] to ZEI_GEN as _gender on $projection.Gender = _gender.Value
9 }

```

Create CDS Entity for Academic Data

1. Right click on node Data Definition and chose New Data Definition.



2. Provide **Name** and **Description** for the Entity and Click **next** . Select **TR** and click on **Next**.

Select Template as '**Define View Entity**' .Finish.

Templates

Select one of the available templates.

☒ Use the selected template

Name	Description
View (creation)	
 defineViewEntity	Define View Entity
 defineRootViewEntity	Define Root View Entity
 defineViewEntityWithToParentAssociation	Define View Entity with To-Parent Association
 defineView	Define View (obsolete as of AS ABAP 7.57)
>  Projection View (creation)	
>  View Extend (creation)	
>  Table Function (creation)	

```
@AbapCatalog.viewEnhancementCategory: [#NONE]
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: '${ddl_source_description}'
@Metadata.ignorePropagatedAnnotations: true
@ObjectModel.usageType:{
  serviceQuality: #X,
  sizeCategory: #S,
  dataClass: #MIXED
}
```

 < Back Next > Finish Cancel

3. Complete Entity with Below code and **Activate Both (ZR_STU , ZI_ACAD)**.

```
[TRL] ZR_STU [TRL] ZI_ACAD x
1 @AbapCatalog.viewEnhancementCategory: [#NONE]
2 @AccessControl.authorizationCheck: #NOT_REQUIRED
3 @EndUserText.label: 'Interface Entity Academic result'
4 @Metadata.ignorePropagatedAnnotations: true
5 @ObjectModel.usageType:{
6   serviceQuality: #X,
7   sizeCategory: #S,
8   dataClass: #MIXED
9 }
10 define view entity ZI_Acad as select from zacademic_tb
11 association to parent ZR_STU as _student on $projection.Id = _student.Id
12 association[1..1] to ZIE_COURSE as _course on $projection.Course = _course.Value
13 association[1..1] to ZIE_SEM as _sem on $projection.Sem = _sem.Value
14 association[1..1] to ZIE_SEMR as _semr on $projection.Semresult = _semr.Value
15 {
16   key id as Id,
17   key course as Course,
18   key sem as Sem,
19   _course.Description as CourseDesc,
20   _sem.Description as SemDesc,
21   semresult as Semresult,
22   _semr.Description as SemRDesc,
23   _student
24 }
25
```

Note : Same Process follow here like Gender Description . Create CDS Entity for Course ,Semester ,Semester Result.

1. Course Entity

```

1 @AbapCatalog.viewEnhancementCategory: [#NONE]
2 @AccessControl.authorizationCheck: #NOT_REQUIRED
3 @EndUserText.label: 'Interface Entity course'
4 @Metadata.ignorePropagatedAnnotations: true
5 @ObjectModel.usageType:{
6   serviceQuality: #X,
7   sizeCategory: #S,
8   dataClass: #MIXED
9 }
10 define view entity ZIE_COURSE as select from DDCDS_CUSTOMER_DOMAIN_VALUE_T
11 ( p_domain_name: 'ZD_COURSE' )
12 {
13   key domain_name,
14   key value_position,
15   @Semantics.language: true
16   key language,
17   value_low as Value,
18   @Semantics.text: true
19   text as Description
20 }
21

```

2. Semester Entity

```

1 @AbapCatalog.viewEnhancementCategory: [#NONE]
2 @AccessControl.authorizationCheck: #NOT_REQUIRED
3 @EndUserText.label: 'Interface Entity Sem'
4 @Metadata.ignorePropagatedAnnotations: true
5 @ObjectModel.usageType:{
6   serviceQuality: #X,
7   sizeCategory: #S,
8   dataClass: #MIXED
9 }
10 define view entity ZIE_SEM as select from DDCDS_CUSTOMER_DOMAIN_VALUE_T
11 ( p_domain_name: 'ZD_SEM' )
12 {
13   key domain_name,
14   key value_position,
15   @Semantics.language: true
16   key language,
17   value_low as Value,
18   @Semantics.text: true
19   text as Description
20 }
21

```

3. Semester result Entity

```

1 @AbapCatalog.viewEnhancementCategory: [#NONE]
2 @AccessControl.authorizationCheck: #NOT_REQUIRED
3 @EndUserText.label: 'Interface Entity Sem result'
4 @Metadata.ignorePropagatedAnnotations: true
5 @ObjectModel.usageType:{
6   serviceQuality: #X,
7   sizeCategory: #S,
8   dataClass: #MIXED
9 }
10 define view entity ZIE_SEMR as select from DDCDS_CUSTOMER_DOMAIN_VALUE_T
11 ( p_domain_name: 'ZD_SEMR' )
12 {
13   key domain_name,
14   key value_position,
15   @Semantics.language: true
16   key language,
17   value_low as Value,
18   @Semantics.text: true
19   text as Description
20 }
21

```

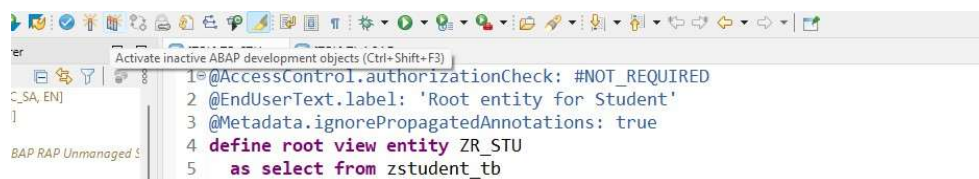
Activate all three Entity . Add in Academic Entity (ZI_ACAD) as Association like below _course,_sem,_semr.


```

1 @AbapCatalog.viewEnhancementCategory: [#NONE]
2 @AccessControl.authorizationCheck: #NOT_REQUIRED
3 @EndUserText.label: 'Interface Entity Academic result'
4 @Metadata.ignorePropagatedAnnotations: true
5 @ObjectModel.usageType:{
6     serviceQuality: #X,
7     sizeCategory: #S,
8     dataClass: #MIXED
9 }
10 define view entity ZI_Acad as select from zacademic_tb
11 association to parent ZR_STU as _student on $projection.Id = _student.Id
12 association[1..1] to ZIE_COURSE as _course on $projection.Course = _course.Value
13 association[1..1] to ZIE_SEM as _sem on $projection.Sem = _sem.Value
14 association[1..1] to ZIE_SEMR as _semr on $projection.Semresult = _semr.Value
15 }

```

4. Activate both the CDS entities together using below option. (**Ctrl + Shift + F3**)



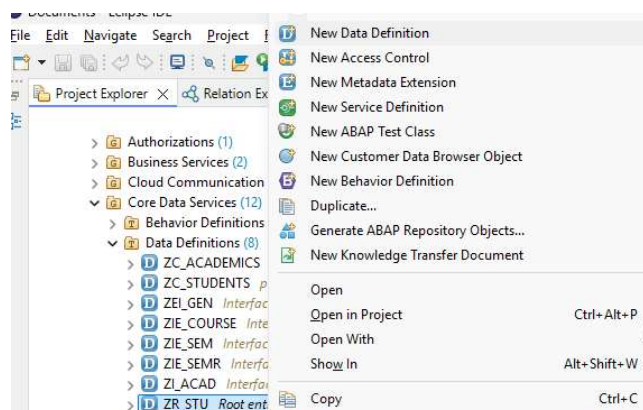
3. Create Projection Entities

Projection Entities define interfaces on the basis of existing CDS Entity. It is always based on exactly one existing CDS view entity and it projects of it's elements required for the business service.

Create new data definition (you already know how to do this)

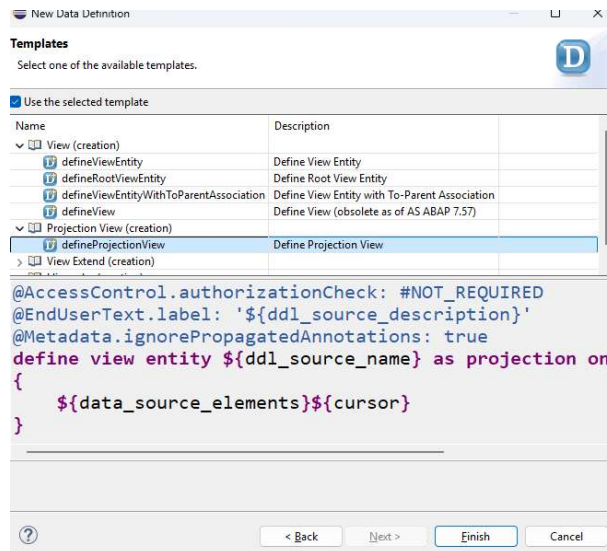
Create Projection For Root Entity Student Data.

1. Right click on root view **CDS > Data Definition**.



2. Enter **Name** and **Description** . Click **Next** . Select **TR** .

3. Choose Template '**Define Projection View**' . Click **Finish**.



4. Complete Entity with below code .

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'projection view for student'
@Metadata.ignorePropagatedAnnotations: true
@Metadata.allowExtensions: true
define root view entity ZC_STUDENTS as projection on ZR_STU
{
  @EndUserText.label: 'StudentID'
  key Id,
  @EndUserText.label: 'ID'
  StudentID,
  @EndUserText.label: 'FirstName'
  Firstname,
  @EndUserText.label: 'LastName'
  Lastname,
  @EndUserText.label: 'AGE'
  Age,
  @EndUserText.label: 'Course'
  Course,
  @EndUserText.label: 'CourseDuration'
  Courseduration,
  @EndUserText.label: 'Status'
  Status,
  @EndUserText.label: 'Gender'
  Gender,
  GenderDesc,
  @EndUserText.label: 'DOB'
  Dob,
  Lastchangedat,
  Locallastchangedat,
  /* Associations */
  _academic : redirected to composition child ZC_ACADEMICS,
  _gender
}
```

Code Explanation :

Key Annotations:

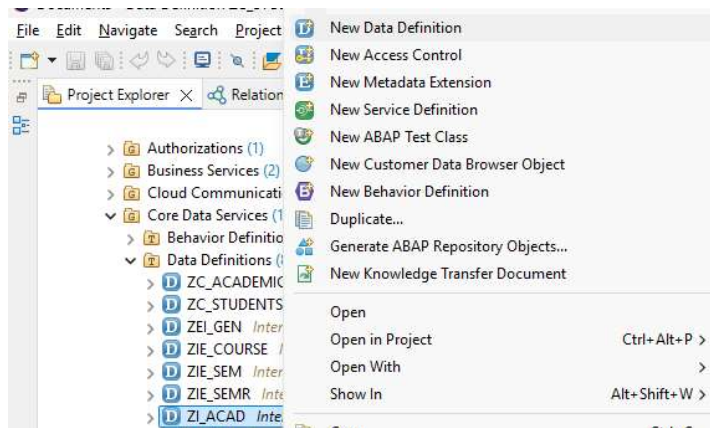
1. **@AccessControl.authorizationCheck: #NOT_REQUIRED:**
 - No authorization check is required for accessing this view.
2. **@EndUserText.label: 'projection view for student':**
 - Provides a user-friendly label for the view, indicating it represents student data.
3. **@Metadata.ignorePropagatedAnnotations: true:**
 - Ensures that annotations from the base entities are not propagated to this view.
4. **@Metadata.allowExtensions: true:**
 - Allows the view to be extended if needed.

Associations:

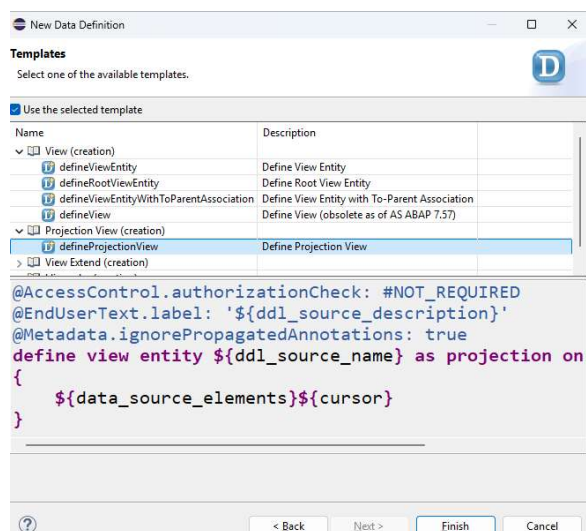
- **_academic : redirected to composition child ZC_ACADEMICS:**
 - This defines an **association** to the academic data (likely grades or courses) of the student, and it is redirected to a composition child entity ZC_ACADEMICS.
- **_gender:**
 - An association to the gender-related data, potentially referencing another entity or attribute related to gender.

Create Projection for Entity Academic Data.

1. follow same process as Student projection entity . only change is that this is not root



2. Enter **Name** and **Description** . Click **Next** . Select **TR** . Choose Template '**Define Projection View**' . Click **Finish**.



3. complete entity with below code.

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'projection view for Academic'
@Metadata.ignorePropagatedAnnotations: true
@Metadata.allowExtensions: true
define view entity ZC_ACADEMICS as projection on ZI_Acad
{
  @EndUserText.label: 'StudentID'
  key Id,
  @EndUserText.label: 'Course'
  key Course,
  @EndUserText.label: 'Semester'
  key Sem,
  CourseDesc,
  SemDesc,
  @EndUserText.label: 'Sem. result'
  Semresult,
  SemRDesc,
  /* Associations */
  _student : redirected to parent ZC_STUDENTS
}
```

4. **Activate** projection entity together.

Code Explanation :

Key Annotations:

1. **@AccessControl.authorizationCheck: #NOT_REQUIRED:**
 - No authorization check is required for accessing this view entity.
2. **@EndUserText.label: 'projection view for Academic':**
 - Label for the view entity, indicating it represents academic data.
3. **@Metadata.ignorePropagatedAnnotations: true:**
 - Ensures that annotations from the base entities (ZI_Acad) are not propagated to this view.
4. **@Metadata.allowExtensions: true:**
 - Allows this view entity to be extended in the future.

Fields:

- **Key Fields:**
 - Id: Student ID (primary key).
 - Course: The course the student is enrolled in (part of the composite key).
 - Sem: The semester (part of the composite key).

Associations:

- **_student : redirected to parent ZC_STUDENTS:**
 - Defines an **association** to the parent view ZC_STUDENTS (student data), which allows for accessing related student information.

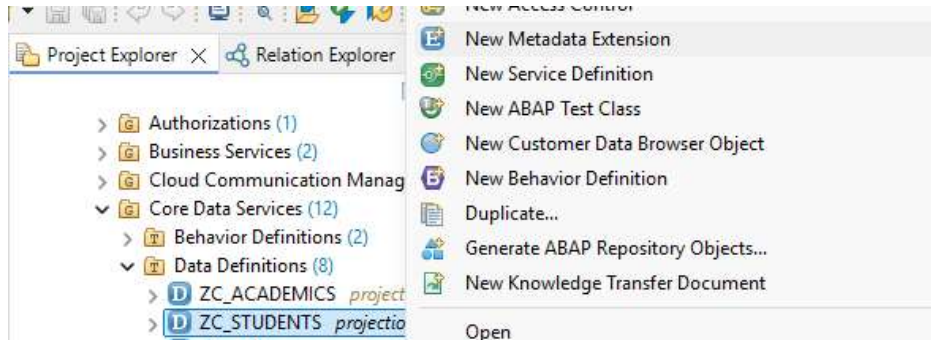
4. Create Metadata Extensions

Metadata Extensions are created to annotate entities at header level and at field level. These annotations drive the descriptions of fields, sequence of fields on application and whether the fields will be available in filter bar or not.

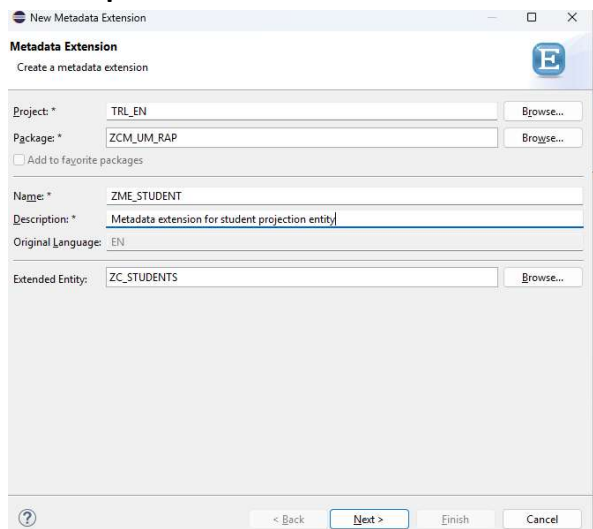
Some of the elements that are highlighted on the screen that are controlled by the metadata extension. The extensions are created at Projection layer.

Create Metadata Extension for ZC_STUDENTS

1. Right click on **Projection Entity**>**New Metadata Extension**.



2. Provide **Name** and **Description** > Select **TR** >**Finish**.



3. Complete entity with below code.

```
@Metadata.layer: #PARTNER
@UI.headerInfo: { typeName: 'Student',           --added page header details
                  typeNamePlural: 'Students',
                  title: { type: #STANDARD ,label: 'Student', value: 'ID' }
                }
```

annotate entity ZC_STUDENTS
with

```
{
  @UI.facet: [ { id : 'Student',                -- to create record need to add facet
                purpose : #STANDARD,
                type : #IDENTIFICATION_REFERENCE,
                label : 'Student',
                position : 10},

  { id:'AcademicResult' ,
    purpose :#STANDARD,
```

```

    type: #LINEITEM_REFERENCE,
    label: 'Academic_Result',
    position: 20,
    targetElement: '_academic'
  } }

@UI:{ lineItem: [{ position: 10, label: 'StudentID' }],
  identification: [{ position: 10, label: 'StudentID' }] }
Id;

  @UI:{ lineItem: [{ position: 20, label: 'ID' }],
  identification: [{ position: 20, label: 'ID' }] }
StudentID;

@UI:{ lineItem: [{ position: 30, label: ' Firstname' }],
  identification: [{ position: 30, label: ' Firstname' }] }
Firstname;

@UI:{ lineItem: [{ position: 40, label: 'Lastname' }],
  identification: [{ position: 40, label: 'Lastname' }] }
Lastname;

@UI:{ lineItem: [{ position: 50, label: 'Age' }],
  identification: [{ position: 50, label: 'Age' }] }
Age;

@UI:{ lineItem: [{ position: 60, label: 'Course' }],
  identification: [{ position: 60, label: 'Course' }] }
Course;

@UI:{ lineItem: [{ position: 70, label: 'Courseduration' }],
  identification: [{ position: 70, label: 'Courseduration' }] }
Courseduration;

@UI:{ lineItem: [{ position: 80, label: ' Status' },
  { type: #FOR_ACTION , dataAction: 'SetAdmitted', label: 'SetAdmitted'}],
  identification: [{ position: 80, label: 'Status' },
  { type: #FOR_ACTION , dataAction: 'SetAdmitted', label: 'SetAdmitted'}] }
Status;

@UI:{ lineItem: [{ position: 90, label: ' Gender' }],
  identification: [{ position: 90, label: ' Gender' }] }

@Consumption.valueHelpDefinition: [{entity:{ name: 'ZEI_GEN' , element: 'Value'}, --annotation for f4 help
  distinctValues :true,

  additionalBinding: [{ localElement: 'GenderDesc',
    element: 'Description' ,
    usage: #FILTER_AND_RESULT}]
  } }

Gender;
@UI:{identification: [{ position: 55, label: '' }] }
GenderDesc;

@UI:{ lineItem: [{ position: 100, label: 'DOB' }],
  identification: [{ position: 100, label: 'DOB' }] }
Dob;

```



```

@UI.hidden: true
Lastchangedat;
@UI.hidden: true
Locallastchangedat;
/* Associations
@UI.hidden: true
_academic;
@UI.hidden: true
_gender;*/ }

```

Code Explanation :

1. Metadata Layer and UI Header:

- The @Metadata.layer: #PARTNER indicates that this metadata is for a partner layer.
- The @UI.headerInfo specifies the display settings for the page header. Here, it's defining the title "Student" with a value of 'ID', and also providing plural forms (e.g., "Students").

2. UI Facets:

- The @UI.facet annotations define groups (facets) that organize the display of fields within the UI.
- One facet is for 'Student', with an IDENTIFICATION_REFERENCE type, and another facet is for 'AcademicResult', which is a line-item reference type linked to an academic element (_academic).

3. Field Annotations:

- Each field (e.g., Id, StudentID, Firstname, Lastname, etc.) is annotated with @UI to define how it should appear in the UI, including:
 - **Line Item:** Specifies how fields should be displayed in lists or tables.
 - **Identification:** Determines how fields are displayed in record views.
 - **Position:** Defines the order in which the fields appear.
- For example, StudentID is labeled as 'StudentID' both in line items and identification sections, with a position specified.

4. Actions:

- The Status field has a FOR_ACTION annotation, which links it to an action (SetAdmitted) that can be triggered in the UI.

5. Value Help (F4 Help):

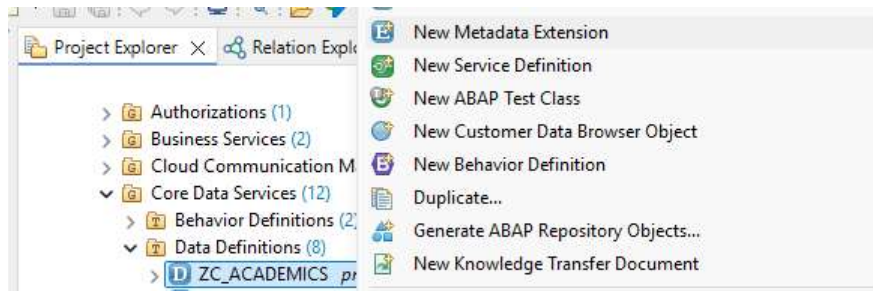
- The @Consumption.valueHelpDefinition annotation provides F4 help (value help) for the Gender field, linking it to an entity ZEI_GEN and providing additional binding for the description.

6. Hidden Fields:

- Fields such as Lastchangedat and Locallastchangedat are marked with @UI.hidden: true, indicating that these fields should be hidden in the UI.

Create Metadata Extension for ZC_ACADEMICS .

1. follow same process like ZC_STUDENTS.



2. Complete entity with below code

@Metadata.layer: #PARTNER

@UI:{

```
  headerInfo: { typeName: 'Academic Result',
                typeNamePlural: 'Academic Result',
                title : { type: #STANDARD,
                        label: 'Academic Result',
                        value: 'ID'
                      },
                description: { type: #STANDARD,
                              label: 'Academic Result',
                              value: 'ID'
                            }
                },
```

```
  presentationVariant : [ { sortOrder : [ { by: 'Sem' , direction: #ASC } ] } ]
}
```

@Search.searchable: true

annotate entity ZC_ACADEMICS

with

{

```
  @UI.facet: [{
    id:'Academic' ,
    purpose :#STANDARD,
    type:#IDENTIFICATION_REFERENCE,
    label: 'Academic Result',
    position: 10 }]
```

```
  @UI:{ lineltem: [{ position: 10, importance: #HIGH,label: 'StudentID' }]}
  @UI.hidden: true
```

Id;

```
  @UI:{ lineltem: [{ position: 20, importance: #HIGH, label: ' Course' } ] ,
    identification: [{ position: 20, label: 'Course' } ] }
```

```
  @Search.defaultSearchElement: true
```

Course;

```
  @UI:{ lineltem: [{ position: 30,importance: #HIGH, label: 'Semister' }],
    identification: [{ position: 30, label: 'Semister' } ] }
```

```
  @Search.defaultSearchElement: true
```

Sem;

```
  @UI:{ lineltem: [{ position: 40,importance: #HIGH, label: '' }],
    identification: [{ position: 40, label: '' } ] }
```

```
  @Search.defaultSearchElement: true
```

```

CourseDesc;
@UI:{ lineltem: [{ position: 50,importance: #HIGH, label: " }},
  identification: [{ position: 50, label: " }]}
SemDesc;
@UI:{ lineltem: [{ position: 60,importance: #HIGH, label: 'Sem Result' }},
  identification: [{ position: 60, label: 'Sem Result' }]}
@Search.defaultSearchElement: true
Semresult;
@UI:{ lineltem: [{ position: 70, importance: #HIGH, label: " }},
  identification: [{ position: 70, label: " }]}
@Search.defaultSearchElement: true
SemRDesc;
/* Associations
@UI.hidden: true
_student;*/
}

```

Code Explanation :

1. Metadata Layer & UI Header:

- The @Metadata.layer: #PARTNER indicates that this is related to the partner layer.
- @UI.headerInfo: Specifies the header details for displaying the academic results, with:
 - typeName: "Academic Result" (singular) and "Academic Result" (plural).
 - The title and description are both set to 'Academic Result' and refer to the 'ID' as their value.

2. Searchable & Presentation Variant:

- @Search.searchable: true: This annotation makes the entity searchable.
- presentationVariant: Defines a sort order for the entity, in this case, sorting by Sem in ascending order (#ASC).

3. Field Annotations:

- @UI.facet: Defines a facet called 'Academic' for grouping fields related to the academic results. It's an identification reference with a label 'Academic Result'.
- Fields like Id, Course, Sem, CourseDesc, SemDesc, Semresult, and SemRDesc are annotated for display in the UI with specific positions and importance.
 - lineltem: Specifies the fields' order and importance in lists or tables.
 - identification: Specifies how these fields will appear when identifying the record.
 - importance: #HIGH: Marks these fields as highly important in the UI.

4. Search-Related Annotations:

- @Search.defaultSearchElement: true: Marks fields like Course, Sem, CourseDesc, Semresult, and SemRDesc as default search elements, making them searchable in the application.

5. Hidden Fields:

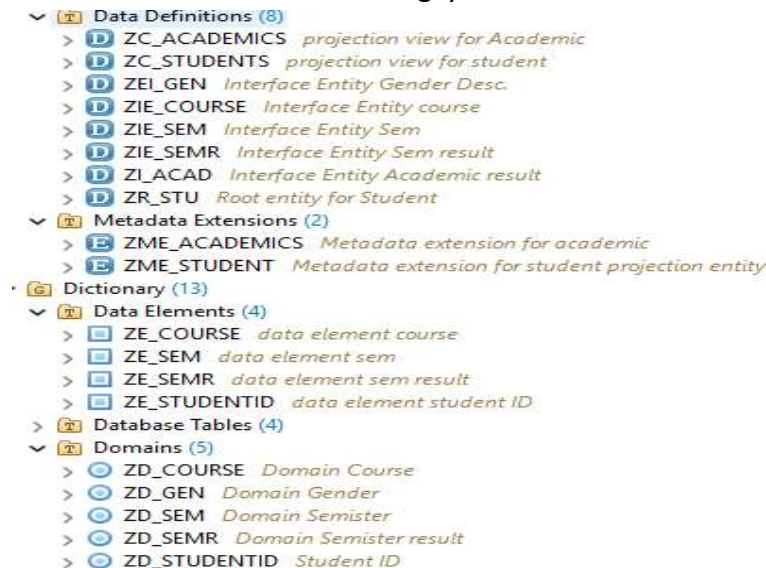
- @UI.hidden: true: The Id field is hidden in the UI.

6. Associations:

- The code has a commented-out line indicating a potential hidden association with a `_student` entity (though it's currently not in use).

3. Activate Both Metadata Extension Together.

Now , the CDS node should look like this assuming you do not have other CDS view.



5. Behavior Definition

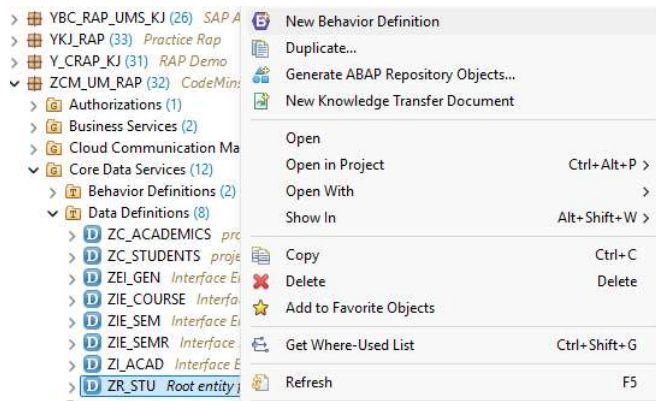
Behavior Definition defines

- What operations (CRUD) are possible for the entities defined.
- Whether the operations are managed by SAP or not managed by SAP i.e. unmanaged. Unmanaged operations are then required to be implemented in the associated class.
- Additional operations/function imports are also implemented in the associated class.
- Read-only fields / Mandatory fields from the service
- Field Mapping

The Behavior Definition is created for the root entity and projected to the consumption layer, however, unlike the CDS Entity projections, both Student and Academic Entities will be included together in one definition.

Create Behavior Definition for Root View.

- Right click on Root View > New Behavior Definition



2. Name of Behavior Definition will be same as **CDS Entity**. Choose **Implementation Type>Next**.

 A screenshot of the 'New Behavior Definition' dialog box. The dialog has the following fields and options:

- Project:** * (Text field) containing 'TRL_EN' and a 'Browse...' button.
- Package:** * (Text field) containing 'ZCM_UM_RAP' and a 'Browse...' button.
- ☐ Add to favorite packages
- Name:** (Text field) containing 'ZR_STU'.
- Description:** * (Text field) containing 'Root entity for Student'.
- Original Language:** (Text field) containing 'EN'.
- Root Entity:** * (Text field) containing 'ZR_STU'.
- Implementation Type:** * (Dropdown menu) showing 'Unmanaged'.
- At the bottom, there are buttons: '< Back', 'Next >', 'Finish', and 'Cancel'.

2. Select the **TR** and choose **Finish** .Notice that the behavior is defined for both entities i.e. **ZR_STUD** and **ZI_ACAD**.

3. Complete Definition with Below Code.

```
unmanaged implementation in class zbp_r_stu unique;
strict ( 2 );
with draft;
```

```
define behavior for ZR_STU alias Student
draft table ZSTUDENT_D_TB
//late numbering
early numbering
lock master
total etag Lastchangedat
authorization master ( instance )
etag master Lastchangedat
{
  create;
  update (features : instance);
  delete (features : instance);
}
```



```
association _academic { create; with draft;}
```

```
draft action Edit;
draft action Activate;
draft action Discard;
draft action Resume;
```

```
draft determine action Prepare;
field ( readonly ) Id , GenderDesc, StudentID;
```

```
mapping for ZSTUDENT_TB {
  Id = id;
  StudentID = studentid;
  Firstname = firstname;
  Lastname = lastname;
  Age = age;
  Course = course;
  Courseduration = courseduration;
  Status = status;
  Gender = gender;
  Dob = dob;
  Lastchangedat = lastchangedat;
  Locallastchangedat = locallastchangedat;
}
}
```

```
define behavior for ZI_Acad alias academic
draft table ZACADEMICS_D_TB
```

```
//late numbering
early numbering
```

```
lock dependent by _student
authorization dependent by _student
etag dependent by _student
```

```
{
  update;
  delete;
  field ( readonly ) Id, Course, Sem;
```

```
association _student{with draft;}
```

```
mapping for zacademic_tb {
  Id = id;
  Course = course;
  Sem = sem;
  Semresult = semresult;
}
}
```

Note : The draft table is required because we will be using an OData V4 service in the service binding. If we use an OData V2 service, there is no need to implement draft functionality, as OData V2 does not support drafts.

The draft functionality is necessary to support features such as editing, saving intermediate states, and managing the lifecycle of entities before final submission.

With OData V4, draft support ensures that changes can be made to entities without affecting the actual data until the draft is activated.

How To Add Draft Functionality :

1. In Behavior Definition add **with Draft** ;

```

E [TRL] ZME_STUDENT  E [TRL] ZME_ACADEMICS  B [TRL] ZR_STU X
1 unmanaged implementation in class zbp_r_stu unique;
2 strict ( 2 );
3 with draft;

```

2. Create draft Table (Root) . Provide Meaningful name>Double click on draft table name

Syntax : **draft table (tablename)**

```

E [TRL] ZME_STUDENT X  E [TRL] ZME_ACADEMICS  B [TRL] ZR_STU X
1 unmanaged implementation in class zbp_r_stu unique;
2 strict ( 2 );
3 with draft;
4
5 define behavior for ZR_STU alias Student
6 draft table ZSTUDENT D TB

```

3. Table look like below and Activate table.

```

E [TRL] ZME_STUDENT  E [TRL] ZME_ACADEMICS  B [TRL] ZR_STU  T [TRL] ZSTUDENT_D_TB X
1 @EndUserText.label : 'Draft table for entity ZR_STU'
2 @AbapCatalog.enhancement.category : #EXTENSIBLE_ANY
3 @AbapCatalog.tableCategory : #TRANSPARENT
4 @AbapCatalog.deliveryClass : #A
5 @AbapCatalog.dataMaintenance : #RESTRICTED
6 define table zstudent_d_tb {
7
8     key mandt      : mandt not null;
9     key id         : sysuuid_x16 not null;
10    studentid      : ze_studentid;
11    firstname       : abap.char(100);
12    lastname        : abap.char(100);
13    age             : abap.char(4);
14    course          : abap.char(50);
15    courseduration  : abap.char(4);
16    status          : abap_boolean;
17    gender          : abap.char(1);
18    genderdesc      : abap.char(60);
19    dob             : abap.dats;
20    lastchangedat   : timestamp;
21    locallastchangedat : timestamp;
22    "%admin"       : include sych_bdl_draft_admin_inc;
23
24 }

```

4. Add below keyword for Draft Table

```

20 draft action Edit;
21 draft action Activate;
22 draft action Discard;
23 draft action Resume;
24
25 draft determine action Prepare;

```

Explanation of draft keywords :

These draft actions are added to the draft table to manage the lifecycle of draft entities in an OData V4 service. They allow users to:

- **Edit:** Modify the draft data before finalizing.
- **Activate:** Commit the changes from the draft to the backend.
- **Discard:** Cancel the draft and discard any changes made.
- **Resume:** Continue working on a previously saved draft.

The **Prepare** action is used to set up an entity for drafting, enabling it to enter the draft state. These actions are necessary to handle temporary data states efficiently before finalizing or discarding changes.

5. make association with Draft

```
17
18 association _academic { create; with draft;}
19
```

6. Now same create draft table for Academic. Provide Meaningful name>Double click on draft table name.

```
44 define behavior for ZI_Acad alias academic
45 draft table ZACADEMICS_D_TB
46
```

7. Table look like below and activate table .

```
[TRL] ZSTUDENT_D_TB [TRL] ZACADEMICS_D_TB X
1 @EndUserText.label : 'Draft table for entity ZI_ACAD
2 @AbapCatalog.enhancement.category : #EXTENSIBLE_ANY
3 @AbapCatalog.tableCategory : #TRANSPARENT
4 @AbapCatalog.deliveryClass : #A
5 @AbapCatalog.dataMaintenance : #RESTRICTED
6 define table zacademics_d_tb {
7
8   key mandt : mandt not null;
9   key id : sysuid_x16 not null;
10  key course : ze_course not null;
11  key sem : ze_sem not null;
12  coursedesc : abap.char(60);
13  semdesc : abap.char(60);
14  semresult : ze_semr;
15  semrdesc : abap.char(60);
16  "%admin" : include sych_bdl_draft_admin_inc;
17
18 }
```

8. make association with Draft

```
3
4
5 association _student{with draft;}
```

Code Explanation :

1. Student Behavior (ZR_STU):

- **Draft Table:** Uses ZSTUDENT_D_TB as the draft table for handling the draft state of student records.
- **Draft Control:** The behavior is controlled by actions like Edit, Activate, Discard, and Resume.
- **Field Control:** The Id, GenderDesc, and StudentID fields are set as read-only (readonly).
- **Actions:** CRUD operations are specified:
 - **Create, Update** (with features for instance), and **Delete** (with features for instance).

- **Associations:**
 - The student record is associated with academic records (_academic).
 - The association is draft-enabled.
- **Authorization & Locking:**
 - Locking is defined on the master record (Student) using lock master.
 - Authorization and ETAG (version control) are handled for master records using etag master and authorization master.
- **Mapping:** Defines the mapping between the ZSTUDENT_TB table and the fields in the ZR_STU entity (Id, StudentID, Firstname, Lastname, etc.).

2. Academic Behavior (ZI_Acad):

- **Draft Table:** Uses ZACADEMICS_D_TB as the draft table for academic records.
- **Field Control:** The fields Id, Course, and Sem are read-only.
- **Actions:**
 - CRUD operations for updating and deleting academic records are allowed.
- **Associations:**
 - The academic records are associated with student records (_student), with draft capabilities enabled.
 - Locking and authorization are defined to be dependent on the student record (lock dependent by _student and authorization dependent by _student).
- **Mapping:** Defines the mapping between the ZACADEMICS_TB table and the fields in the ZI_Acad entity (Id, Course, Sem, Semresult).

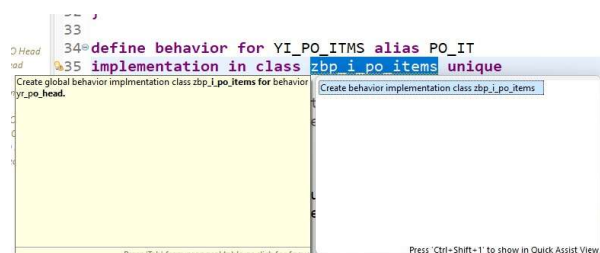
3. Draft and Locking:

- Both entities implement draft behavior, which helps in maintaining intermediate states before final commits.
- Locks and version control (ETAG) are set up for both entities to ensure data consistency.

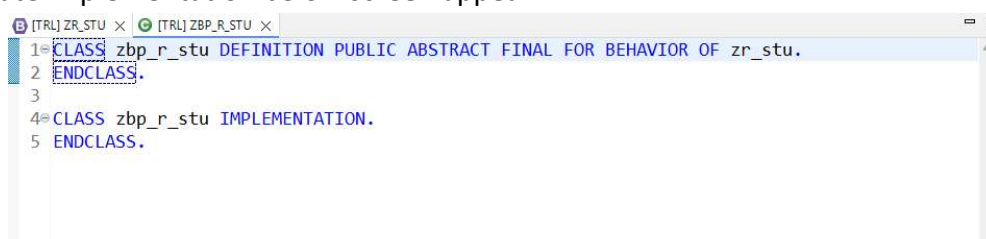
4. Implement the class by **double clicking** on Define view same for ZI_ACAD.(note : first need to activate Behavior definition)



Note : below is just example we need to do like this



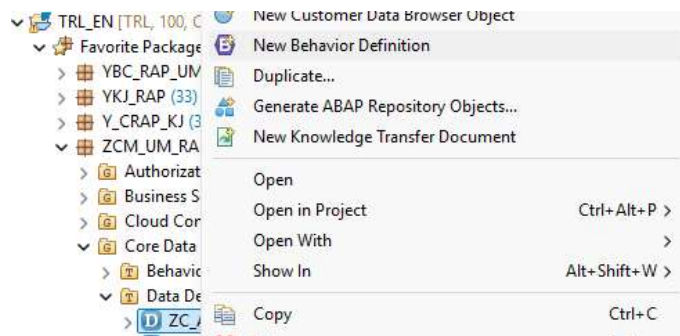
5. When you create implementation below screen appear.



6. Activate Class.

Create Behavior Definition for Projection Root View

1. Right click on **Projection Root View** > **New Behavior Definition**.



2. click **Next** and **Finish**.

3. Complete root Projection By below Code.

```
projection;
strict ( 2 );
use draft;
```

```
define behavior for ZC_STUDENTS alias student
```

```
{
  use create;
  use update;
  use delete;
```

```
  use action Edit;
  use action Activate;
  use action Discard;
  use action Resume;
  use action Prepare;
```

```
  use association _academic { create; with draft; }
}
```



```

define behavior for ZC_ACADEMICS alias academic
{
    use update;
    use delete;

    use association _student { with draft; }
}

```

Code Explanation :

This code defines the behavior for two entities, ZC_STUDENTS (Student) and ZC_ACADEMICS (Academic), in a draft-enabled OData service. Here's a short explanation:

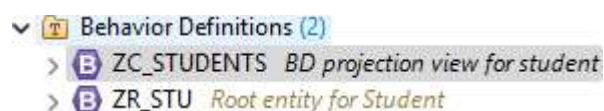
1. ZC_STUDENTS Behavior (Student):

- **CRUD Operations:** The entity allows **create**, **update**, and **delete** operations for student records.
- **Draft Actions:** Actions like **Edit**, **Activate**, **Discard**, **Resume**, and **Prepare** are enabled to manage draft entities. These actions help with editing, finalizing, discarding, resuming, and preparing student records in draft mode.
- **Association:** An association with the ZC_ACADEMICS (academic) entity is included, allowing the creation and handling of academic records in draft mode.

2. ZC_ACADEMICS Behavior (Academic):

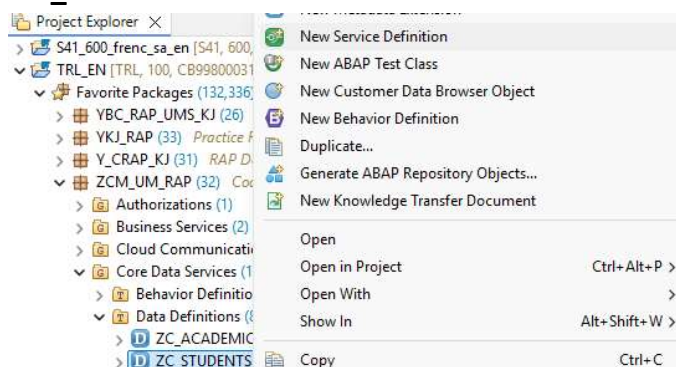
- **Update and Delete:** The academic entity supports **update** and **delete** operations.
- **Association:** There is an association with the ZC_STUDENTS (student) entity, allowing it to reference student records in draft mode.

Note : Behavior Definition should be like Below.



5. Service Definition

1. Right click on **ZC_STUDENTS** and choose **New Service Definition**.



2. Enter **Name** and **Description** . Click **Next** . Select **TR** . Click **Finish**.

Service Definition
Create a service definition

Project: *

Package: *

☐ Add to favorite packages

Name: *

Description: *

Original Language:

Source Type:

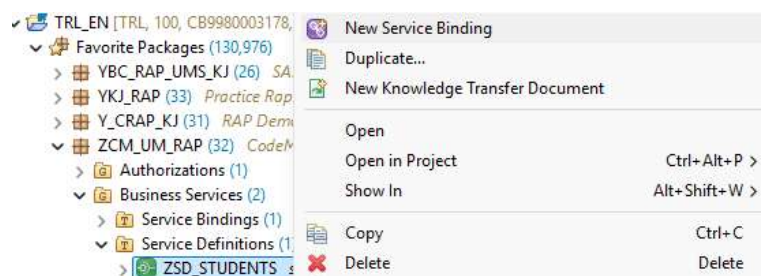
Referenced Object:

3. Complete Service definition by below code.

```
[TRL] ZSD_STUDENTS x
1 @EndUserText.label: 'service definition for student'
2 define service ZSD_STUDENTS {
3   expose ZC_STUDENTS as student;
4   expose ZC_ACADEMICS as academic;
5 }
```

6. Service Binding

1. Right click on **Service Definition>New Service Binding**.



2. Enter **Name** and **Description** , **Binding Type** . Click **Next** . Select **TR** . Click **Finish**.

(We are using the service binding type OData V4 because we are utilizing draft functionality here.)

New Service Binding

Specify a value for field 'Binding Type'

Project: *

Package: *

☐ Add to favorite packages

Name: *

Description: *

Original Language:

Binding Type: *

Service Definition: *

3. Activate Binding then click on **ZSD_Students > Publish**.

Service Binding: ZSB_STUDENTS

General Information
This section describes general information about this service binding

Binding Type: OData V4 - UI

Services
Define services associated with the Service Binding

Default Authorization Values: CE3EABB58D007057AD39A866ABDC98HT

Local Service Endpoint:

type filter text

Service ...	Version	API ...	Service Definition
✓ ZSD_STU	1.0.0	Not ...	ZSD_STUDENTS

4. To see fiori application click on **Student > preview**.

Service Binding: ZSB_STUDENTS

General Information
This section describes general information about this service binding

Binding Type: OData V4 - UI

Services
Define services associated with the Service Binding

Default Authorization Values: CE3EABB58D007057AD39A866ABDC98HT

Local Service Endpoint:

type filter text

Service ...	Version	API ...	Service Definition
✓ ZSD_STU	1.0.0	Not ...	ZSD_STUDENTS

Service Version Details
View information on selected service version

Service Information
Service URL:

type filter text

Entity Set and Association
Open Fiori Elements App Preview for selected entity set or association

academic student student _academic

Fiori Preview Look like Below.

Standard

Editing Status:

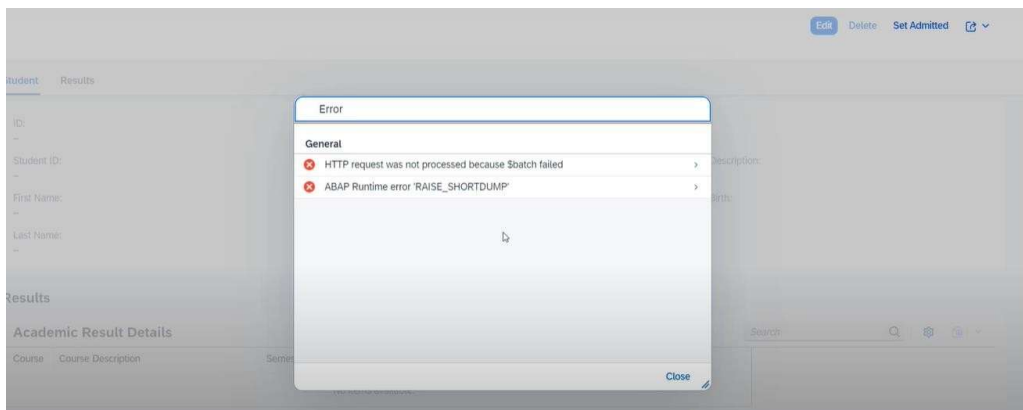
Adapt Filters (1)

Students

StudentID	ID	Firstname	Lastname	Age	Course	Coursedura...
☐						

To start, set the relevant filters and choose "Go".

Note : While creating a record, we encounter an error, specifically a short dump. This happens because, in an unmanaged scenario, we need to write code for each operation we want to perform in the application.

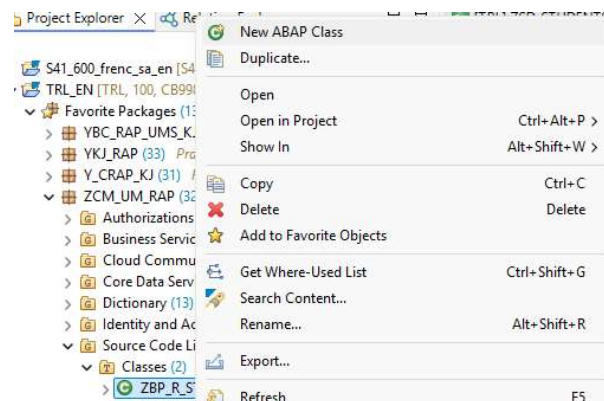


Note :

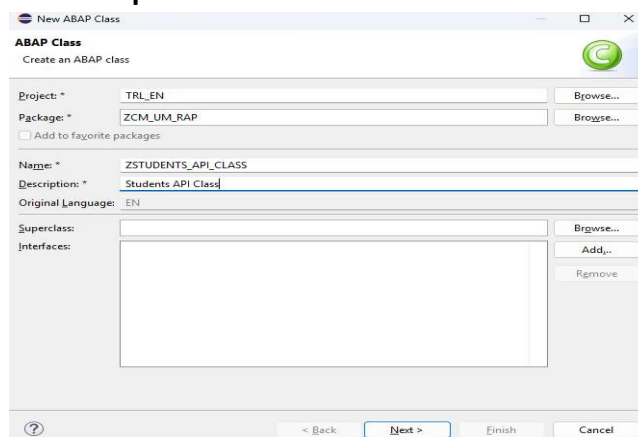
- Until now, we haven't implemented the application, meaning we haven't written the code for CRUD operations. Let's now begin the main work by creating the necessary classes and methods that we will use throughout the application. The first step is to create a API class that will contain all of our methods, which will handle the **Create, Read, Update, and Delete** functionalities.
- Depending on your preference, you can either create a separate class (e.g., **ZSTUDENT_API_CLASS**) where you can implement the CRUD operations and then call those methods in the main class (**ZBP_R_STU**), or you can directly write the CRUD operations in the class you created during the behavior definition (**ZBP_R_STU**).

1. Create class

1. right click on **classes > New ABAP Class** .



2. Provide **Name and Description**. Click **Next** > Select **TR** and **Finish**.



3. Class Template look like below Our Class Name is **ZSTUDENTS_API_CLASS** (below template is for example)

```

1 CLASS zcl_students_api_class DEFINITION
2   PUBLIC
3   FINAL
4   CREATE PUBLIC .
5
6   PUBLIC SECTION.
7   PROTECTED SECTION.
8   PRIVATE SECTION.
9   ENDClass.
10
11
12
13 CLASS zcl_students_api_class IMPLEMENTATION.
14 ENDClass.

```

Early Unmanaged Numbering

- Early Numbering is Implemented by Adding **EARLY NUMBERING** keyword in Behavior Definition of Entity
- Automatically assign a key to Business Object Entity Instance during Methods **Create / Create By Association**
- Keys are generated in **INTERACTION PHASE** of RAP

```

12
13   METHODS earlynumbering_create FOR NUMBERING
14   IMPORTING entities FOR CREATE Student.
15
34   METHODS earlynumbering_cba_Results FOR NUMBERING
35   IMPORTING entities FOR CREATE Student\Results.
36

```

1. Below code is For Early Numbering And Create data and Save.

Step1. In Main class **zstudent_api_class**. Complete class by below code.

```

CLASS zstudents_api_class DEFINITION
PUBLIC
FINAL
CREATE PUBLIC .

```

```

PUBLIC SECTION.

```

```

TYPES : tt_create_student TYPE TABLE FOR CREATE zr_stu\\Student,
        tt_mapped_early  TYPE RESPONSE FOR MAPPED EARLY zr_stu,
        tt_response_early TYPE RESPONSE FOR FAILED EARLY zr_stu,
        tt_reported_early TYPE RESPONSE FOR REPORTED EARLY zr_stu,
        tt_reported_late  TYPE RESPONSE FOR REPORTED LATE zr_stu.

```

"create Constructor

```

CLASS-METHODS: get_instance RETURNING VALUE(ro_instance) TYPE REF TO zstudents_api_class.

```

"class methods

METHODS:

"early number method

earlynumbering_create_student

```

IMPORTING entities TYPE tt_create_student "table for create zr_stu\\student
CHANGING mapped TYPE tt_mapped_early "response for mapped early zr_stu
failed TYPE tt_response_early "response for failed early zr_stu
reported TYPE tt_reported_early, "response for reported early zr_stu

```

"create method

create_student

```

IMPORTING entities TYPE tt_create_student "table for create zr_stu\\student
CHANGING mapped TYPE tt_mapped_early "response for mapped early zr_stu
failed TYPE tt_response_early "response for failed early zr_stu
reported TYPE tt_reported_early, "response for reported early zr_stu

```

"save method

savadata

```

CHANGING reported TYPE tt_reported_late, "response for reported late zr_stu .

```

```
get_next_id
  RETURNING VALUE(rv_id) TYPE string,
```

```
get_next_student_id
  RETURNING VALUE(rv_studentid) TYPE i.
```

PROTECTED SECTION.

PRIVATE SECTION.

```
CLASS-DATA: mo_instance TYPE REF TO zstudents_api_class,
  gt_student TYPE STANDARD TABLE OF zstudent_tb,
  gs_mapped TYPE tt_mapped_early,
  gt_result TYPE STANDARD TABLE OF zacademic_tb.
```

```
ENDCLASS
```

CLASS zstudents_api_class IMPLEMENTATION.

METHOD get_instance.

```
mo_instance = ro_instance = COND #( WHEN mo_instance IS BOUND
  THEN mo_instance
  ELSE NEW #( ) ).
```

ENDMETHOD.

METHOD get_next_id.

```
TRY.
  rv_id = cl_uuid_factory=>create_system_uuid( )->create_uuid_x16( ).
CATCH cx_uuid_error.
  "handle exception
```

ENDTRY.

ENDMETHOD.

METHOD get_next_student_id.

```
SELECT MAX( studentid ) FROM zstudent_tb INTO @DATA(lv_max_studentid).
rv_studentid = lv_max_studentid + 1.
```

ENDMETHOD.

METHOD earlynumbering_create_student.

```
DATA(ls_mapped) = gs_mapped.
" DATA(lv_new_id) = cl_uuid_factory=>create_system_uuid( )->create_uuid_x16( ).
```

```
DATA(lv_new_id) = get_next_id( ).
```

"buffer table update

```
READ TABLE gt_student ASSIGNING FIELD-SYMBOL(<lfs_students>) INDEX 1.
IF <lfs_students> IS ASSIGNED.
  <lfs_students>-id = lv_new_id.
  UNASSIGN <lfs_students>.
ENDIF.
```

```
mapped-student = VALUE #(
  FOR ls_entities IN entities WHERE ( id IS INITIAL )
  (
    %cid = ls_entities-%cid
    %is_draft = ls_entities-%is_draft
    id = lv_new_id ) ).
```

ENDMETHOD.

METHOD create_student. "to save data coming from frontend to backend and saving/sending in persistant table


```
gt_student = CORRESPONDING #( entities MAPPING FROM ENTITY ).
```

```
"new syntax
```

```
IF NOT gt_student[] IS INITIAL.
```

```
  gt_student[ 1 ]-studentid = get_next_student_id( ).
```

```
ENDIF.
```

```
mapped = VALUE #(
```

```
  student = VALUE #(
```

```
    FOR ls_entity IN entities (
```

```
      %cid = ls_entity-%cid
```

```
      %key = ls_entity-%key
```

```
      %is_draft = ls_entity-%is_draft
```

```
    )
```

```
  )
```

```
).
```

```
"Old Syntax
```

```
*
```

```
* LOOP AT entities ASSIGNING FIELD-SYMBOL(<lfs_entities>).
```

```
* IF NOT gt_student[] IS INITIAL.
```

```
*   gt_student[ 1 ]-studentid = get_next_student_id( ).
```

```
*   mapped-student = VALUE #(
```

```
*       ( %cid = <lfs_entities>-%cid
```

```
*       %key = <lfs_entities>-%key
```

```
*       %is_draft = <lfs_entities>-%is_draft )
```

```
*   ).
```

```
* ENDIF.
```

```
* ENDLOOP.
```

```
ENDMETHOD.
```

```
METHOD savedata.
```

```
  IF NOT gt_student[] IS INITIAL.
```

```
    MODIFY zstudent_tb FROM TABLE @gt_student.
```

```
  ENDIF.
```

```
ENDMETHOD.
```

```
ENDCLASS.
```

Code Explanation :

This ABAP class, `zstudents_api_class`, handles student data processing in an SAP system, with a focus on managing draft records and creating new student entries. Here's a breakdown of the key components:

1. Types:

Defines tables for handling different responses related to student data, such as `tt_create_student` (for student creation), `tt_mapped_early`, `tt_response_early`, `tt_reported_early`, and `tt_reported_late`, which are used for different stages of the draft process (early, late, etc.).

2. Class Methods:

- **get_instance:** Ensures a singleton instance of the class is returned (uses COND to create or reuse the instance).
- **get_next_id:** Generates a new unique ID using the `cl_uuid_factory` class.
- **get_next_student_id:** Retrieves the next available student ID by finding the maximum current student ID in the `zstudent_tb` table and incrementing it.
- **earlynumbering_create_student:** Creates a draft for a new student, assigns a unique ID, and updates the draft table (`gt_student`).

- **create_student:** Handles creating student records from frontend data, mapping the input data, generating a student ID, and preparing the data for persistence.
- **savedata:** Saves the data from the draft (gt_student) to the persistent zstudent_tb table.

3. Private Section:

- Defines internal data structures like gt_student (a table of students), gs_mapped (for mapped draft data), and gt_result (for academic data).

Key Concepts:

- **Draft Handling:** The class includes methods to handle drafts (earlynumbering_create_student, create_student) and manage the creation of student records in draft mode before they are finalized and saved.
- **UUID for ID Generation:** New student IDs are generated using the cl_uuid_factory class to ensure uniqueness.
- **Persistence:** Data from the frontend (via create_student) is saved into the zstudent_tb database table.

Step2.In Behavior Implementation class **ZBP_R_STU** Call here API Created method.

Add below code in **CLASS lhc_Student IMPLEMENTATION**.

```

46
47 METHOD create.
48
49     zstudents_api_class=>get_instance( )->create_student(
50         EXPORTING
51             entities = entities
52         CHANGING
53             mapped = mapped
54             failed = failed
55             reported = reported
56     ).
57 ENDMETHOD.
58
59 METHOD earlynumbering_create.
60
61     zstudents_api_class=>get_instance( )->earlynumbering_create_student(
62         EXPORTING
63             entities = entities
64         CHANGING
65             mapped = mapped
66             failed = failed
67             reported = reported
68     ).
69 ENDMETHOD.

```

Code explanation :

In the provided code, the methods **create** and **earlynumbering_create** are invoking the class methods from zstudents_api_class to handle student data. Here's a breakdown:

1. create Method:

- This method calls the create_student class method from zstudents_api_class.
- **Purpose:** It is used to create a student record in the system.
- **Parameters:**
 - entities: The input data for creating the student, likely passed from a frontend or service layer.
 - mapped: This is a changing parameter that will hold the mapped data after processing.
 - failed: This holds information about any errors or failed creation attempts.

- reported: This stores the response for reported student records.
- The method passes these parameters to the `create_student` method, which processes the creation and updates the values of mapped, failed, and reported.

2. earlynumbering_create Method:

- This method calls the `earlynumbering_create_student` class method from `zstudents_api_class`.
- Purpose:** It handles the creation of a draft student record, including assigning an early unique ID to the student.
- Parameters:**
 - entities: The input data for the student draft.
 - mapped: A changing parameter that will store the mapped early student data.
 - failed: This will hold the details of any failed draft creation attempts.
 - reported: This stores the response for reported early draft records.
- Similar to `create`, the method passes these parameters to the `earlynumbering_create_student` method, which processes the draft creation, assigns IDs, and updates mapped, failed, and reported.

Add below code in save class : **CLASS lsc_ZR_STU IMPLEMENTATION.**

```

147 METHOD check_before_save.
148 ENDMETHOD.
149
150 METHOD save.
151
152     zstudents_api_class=>get_instance( )->savedata(
153         CHANGING
154         reported = reported
155     ).
156
157 ENDMETHOD.

```

Code explanation :

The **save** method calls the **savedata** method from the `zstudents_api_class` to save student data to the backend.

Explanation:

- zstudents_api_class=>get_instance()->savedata:**
 - This calls the `savedata` method of the `zstudents_api_class` class, which saves the student data (from the draft table `gt_student`) to the persistent `zstudent_tb` table.
- CHANGING reported = reported:**
 - The `reported` parameter is passed by reference. The method will update `reported` with the result of the save operation (e.g., success or failure response).

Activate All Entities > Preview application

1. Click on **Create** .

Editing Status:
All

Go Adapt Filters (1)

Students

SetAdmitted Create Delete

Ctrl+Enter

StudentID	ID	Firstname	Lastname	Age	Course
To start, set the relevant filters and choose "Go".					

2. Screen will appear like below . Now we are able to create record.

c6ac816a-af1d-1edf-b69d-516efceb8329 SetAdmitted

Student Academic_Result

StudentID: c6ac816a-af1d-1edf-b69d-516efceb8329

Age:

Status: ☐

ID: -

Short Descript.: -

Gender:

Firstname:

Course:

DOB: e.g. Dec 31, 2025

Lastname:

Courseduration:

Academic_Result

Search

Create Delete

Course	Semister	Short Descript.	Short Descript.	Sem Result	Short Descript.
No items available.					

Create Discard Draft

3. Fill record and in bottom corner is **show draft updated** > click on **create** .

c6ac816a-af1d-1edf-b69d-516efceb8329 SetAdmitted

Student Academic_Result

StudentID: c6ac816a-af1d-1edf-b69d-516efceb8329

Age: 23

Status: ☐

ID: -

Short Descript.: Female

Gender: F

Firstname: Divya

Course: SAP Basis

DOB: e.g. Dec 31, 2025

Lastname: Deshmukh

Courseduration: 4

Academic_Result

Search

Create Delete

Course	Semister	Short Descript.	Short Descript.	Sem Result	Short Descript.
No items available.					

Draft updated Create Discard Draft

4. Record created **successfully** .

c6ac816a-af1d-1edf-b69d-5aa35e5ca32f Edit Delete SetAdmitted

Ctrl+E

Student Academic_Result

StudentID: c6ac816a-af1d-1edf-b69d-5aa35e5ca32f

Age: 23

Status: No

ID: 3

Short Descript.: Female

Gender: F

Firstname: Divya

Course: SAP Basis

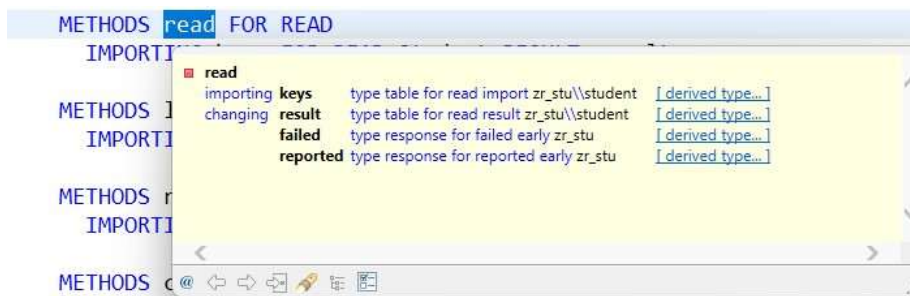
DOB: -

Lastname: Deshmukh

Courseduration: 4

2. Read Record :

Step1.Click F2 on Read Method of Main class . **Copy** and **paste** into Student API Class.



Step2.Create types of importing keys and result and Changing just replace into Methods read by tt_student_keys.

```
tt_student_keys TYPE TABLE FOR READ IMPORT zr_stu\\student,
tt_student_result TYPE TABLE FOR READ RESULT zr_stu\\student,
```

```
"read method
read_student
IMPORTING keys TYPE tt_student_keys "table for READ IMPORT zr_stu\\student
CHANGING result TYPE tt_student_result "table for READ result zr_stu\\student
failed TYPE tt_response_early "response for failed early zr_stu
reported TYPE tt_reported_early "response for reported early zr_stu
```

Step3.Implement the read method

```
METHOD read_student.
  SELECT * FROM zstudent_tb FOR ALL ENTRIES IN @keys
  WHERE id = @keys-id
  INTO TABLE @DATA(lt_student_data).

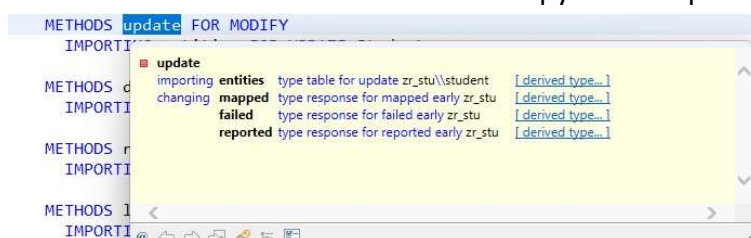
  result = CORRESPONDING #( lt_student_data MAPPING TO ENTITY ).
ENDMETHOD.
```

Step4.Call Read Method into Main class And **Activate**.

```
METHOD read.
  zstudents_api_class=>get_instance( )->read_student(
    EXPORTING
      keys = keys
    CHANGING
      result = result
      failed = failed
      reported = reported
  ).
ENDMETHOD.
```

3. Update Record :

Step1.Click F2 on Read Method in our main class . Copy this and paste into Student API Class



Step2. Need to create type of importing update and for Changing just replace and add in methods update by tt_student_update .

```
tt_student_update TYPE TABLE FOR UPDATE zr_stu\\student,
```

```
"update method
update_student
  IMPORTING entities TYPE tt_student_update "table for update zr_stu\\student [ derived type... ]
  CHANGING mapped TYPE tt_mapped_early "response for mapped early zr_stu [ derived type... ]
          failed TYPE tt_response_early "response for failed early zr_stu [ derived type... ]
          reported TYPE tt_reported_early, "response for reported early zr_stu [ derived type... ]
  .
```

Step3. Implement Update method .

METHOD update_student.

```
DATA : lt_student_update TYPE STANDARD TABLE OF zstudent_tb,
      lt_student_update_x TYPE STANDARD TABLE OF zstr_student.
```

```
lt_student_update = CORRESPONDING #( entities MAPPING FROM ENTITY ).
```

```
lt_student_update_x = CORRESPONDING #( entities MAPPING FROM ENTITY USING CONTROL ).
```

```
IF NOT lt_student_update IS INITIAL.
```

```
  SELECT * FROM zstudent_tb
  FOR ALL ENTRIES IN @lt_student_update
  WHERE id = @lt_student_update-id
  INTO TABLE @DATA(lt_student_update_old).
ENDIF.
```

```
gt_student = VALUE #(
```

```
  FOR x = 1 WHILE x <= lines( lt_student_update )
  LET
    ls_control_flag = VALUE #( lt_student_update_x[ x ] OPTIONAL )
    ls_student_new = VALUE #( lt_student_update[ x ] OPTIONAL )
    ls_student_old = VALUE #( lt_student_update_old[ id = ls_student_new-id ] OPTIONAL )
```

```
  IN
  (
    id = ls_student_new-id
    studentid = COND #( WHEN ls_control_flag-studentid IS NOT INITIAL
                        THEN ls_student_new-studentid ELSE ls_student_old-studentid )
    firstname = COND #( WHEN ls_control_flag-firstname IS NOT INITIAL
                        THEN ls_student_new-firstname ELSE ls_student_old-firstname )
    lastname = COND #( WHEN ls_control_flag-lastname IS NOT INITIAL
                        THEN ls_student_new-lastname ELSE ls_student_old-lastname )
    age = COND #( WHEN ls_control_flag-age IS NOT INITIAL
                  THEN ls_student_new-age ELSE ls_student_old-age )
    course = COND #( WHEN ls_control_flag-course IS NOT INITIAL
                     THEN ls_student_new-course ELSE ls_student_old-course )
    courseduration = COND #( WHEN ls_control_flag-courseduration IS NOT INITIAL
                              THEN ls_student_new-courseduration ELSE ls_student_old-courseduration )
    Status = COND #( WHEN ls_control_flag-Status IS NOT INITIAL
                     THEN ls_student_new-Status ELSE ls_student_old-Status )
    gender = COND #( WHEN ls_control_flag-gender IS NOT INITIAL
                     THEN ls_student_new-gender ELSE ls_student_old-gender )
    dob = COND #( WHEN ls_control_flag-dob IS NOT INITIAL
                  THEN ls_student_new-dob ELSE ls_student_old-dob )
  ).
```

```
ENDMETHOD.
```

Step4. Call method in main class And **Activate**.

METHOD `update.`

```
zstudents_api_class=>get_instance( )->update_student(
  EXPORTING
    entities = entities
  CHANGING
    mapped = mapped
    failed = failed
    reported = reported
).
```

4. Create by Association (Child Entity)

When we create associations between entities, it typically implies that we're working with a child entity in the context of an object-oriented design or data model in ABAP. In such scenarios, there are certain key operations and methods that are required for the child entity's lifecycle management. Specifically:

1. CDA Method (Create by Association):

- This method is responsible for creating the child entity records in the database, based on the association. It allows the main entity (the parent) to create and link a new child entity through the association.

2. RBA Method (Read by Association):

- This method facilitates reading or retrieving the associated child entity records. It uses the relationship (or association) between the parent and child entities to fetch the relevant data. It ensures that when the parent entity is fetched, its associated child entities can be accessed as well.

3. Early Numbering (EarlyNumbering_CBA Method):

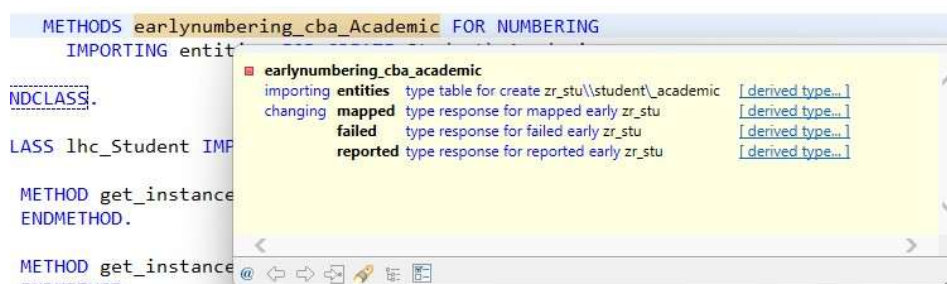
- The EarlyNumbering_CBA method is typically used for generating or managing a unique identifier (number) for child entity records. This might involve generating sequential or system-defined numbers that are used as identifiers for the child entity records. This numbering method could be critical in cases where child records need to be uniquely identified, such as for billing or transactional systems.

```
METHODS rba_Academic FOR READ
  IMPORTING keys_rba FOR READ Student\Academic FULL result_requested RESULT result LINK association_links.
```

```
METHODS cba_Academic FOR MODIFY
  IMPORTING entities_cba FOR CREATE Student\Academic.
```

1. Earlynumbering_cba (For Numbering)

Step1. Click **f2** on Earlynumbering_cba , Copy and Paste into Student API class.



Step2. Create Type for Association and Add into methods.

Type:

```
tt_cba_academic TYPE TABLE FOR CREATE zr_stu\\student\_academic,
```

Method:

```
"earlynumbering_cba_academic
earlynumbering_cba_academic
IMPORTING entities TYPE tt_cba_academic "table for create zr_stu\\student\_academic [ derived type... ]
CHANGING mapped TYPE tt_mapped_early "response for mapped early zr_stu [ derived type... ]
         failed TYPE tt_response_early "for failed early zr_stu [ derived type... ]
         reported TYPE tt_reported_early, "response for reported early zr_stu [ derived type... ]
```

Step3.Implement association of numbering in **Student API Class**.

```
METHOD earlynumbering_cba_academic.
  DATA(lv_new_result_id) = get_next_id( ).

  LOOP AT entities ASSIGNING FIELD-SYMBOL(<lfs_entities>).
    LOOP AT <lfs_entities>-%target ASSIGNING FIELD-SYMBOL(<lfs_entities_create>).
      mapped-academic = VALUE #( (
        %cid = <lfs_entities_create>-%cid
        %key = <lfs_entities_create>-%key
        %is_draft = <lfs_entities_create>-%is_draft ) ).
    ENDLOOP.
  ENDLOOP.
ENDMETHOD.
```

Step4.Call method into Main class And Activate.

```
METHOD earlynumbering_cba_Academic.
  zstudents_api_class=>get_instance( )->earlynumbering_cba_academic(
    EXPORTING
      entities = entities
    CHANGING
      mapped = mapped
      failed = failed
      reported = reported
  ).
ENDMETHOD.
```

2. Cba (create record)

Step1.Click f2 on CBA and Copy and Paste code in Student API class.

Step2. Types already created just replace it.

Method :

```
"cba_academic
cba_academic
IMPORTING entities_cba TYPE tt_cba_academic "table for create zr_stu\\student\_academic [ derived type... ]
CHANGING mapped TYPE tt_mapped_early "response for mapped early zr_stu [ derived type... ]
         failed TYPE tt_response_early "response for failed early zr_stu [ derived type... ]
         reported TYPE tt_reported_early, "response for reported early zr_stu [ derived type... ]
```

3. Implement CBA.

```

METHOD cba_academic.
  gt_result = VALUE #(
    FOR ls_entities_cba IN entities_cba
    FOR ls_result_cba IN ls_entities_cba-%target
    LET
      ls_rap_result = CORRESPONDING zacademic_tb(
        ls_result_cba MAPPING FROM ENTITY
      )
    IN ( ls_rap_result )
  ).
  mapped-academic = VALUE #(
    FOR i = 1 WHILE i <= lines( entities_cba )
    LET
      lt_results = VALUE #( entities_cba[ i ]-%target OPTIONAL )
    IN
      FOR j = 1 WHILE j <= lines( lt_results )
      LET
        ls_curr_result = VALUE #( lt_results[ j ] OPTIONAL )
      IN
        ( %cid = ls_curr_result-%cid
          %key = ls_curr_result-%key
          id = ls_curr_result-Id
        )
      ).
    ).
ENDMETHOD.

```

Step4.Call CBA method into Main Class And **Activate**.

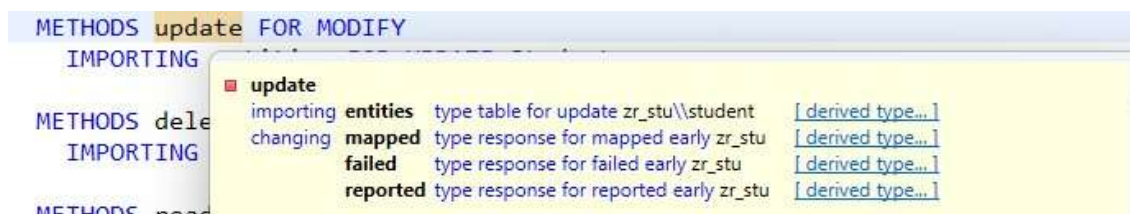
```

METHOD cba_Academic.
  zstudents_api_class=>get_instance( )->cba_academic(
    EXPORTING
      entities_cba = entities_cba
    CHANGING
      mapped = mapped
      failed = failed
      reported = reported
  ).
ENDMETHOD.

```

5.Delete Record

Step1.Click **F2** on update method and Copy ,paste into Student Api class.



Step2.Create Type of importing delete and changing just replace it.

Type :

```
tt_student_delete TYPE TABLE FOR DELETE zr_stu\\student.
```

Method :

```

" delete
delete
IMPORTING keys      TYPE tt_student_delete "table for delete zr_stu\\student [ derived type... ]
CHANGING mapped    TYPE tt_mapped_early  "response for mapped early zr_stu [ derived type... ]
          failed    TYPE tt_response_early "response for failed early zr_stu [ derived type... ]
          reported  TYPE tt_reported_early ,"response for reported early zr_stu [ derived type... ]

```

Step3.**Implement** Delete method.

```

METHOD delete.
  DATA : lt_student TYPE STANDARD TABLE OF zstudent_tb.
  lt_student = CORRESPONDING #( keys MAPPING FROM ENTITY ).
  gr_student_d = VALUE #(
    FOR ls_student_d IN lt_student
      sign = 'I'
      option = 'EQ'
      ( low = ls_student_d-id )
  ).
ENDMETHOD.

```

Step4.**Call** delete method in Main class And **Activate**.

```

METHOD delete.
  zstudents_api_class=>get_instance( )->delete(
    EXPORTING
      keys = keys
    CHANGING
      mapped = mapped
      failed = failed
      reported = reported
  ).
ENDMETHOD.

```